AgentISA: Formal Instruction Set Design for LLM Agents

Anonymous Author(s)

Abstract

Large Language Models are increasingly deployed as autonomous agents, yet their behavior is orchestrated through ad-hoc prompt engineering and per-provider function calling, making it difficult to specify, analyze, and optimize systematically. We present AgentISA, an instruction set architecture that treats agent behavior as a first-class compilable artifact: (1) a formal instruction set with compositional semantics, a type system, and pre/post-conditions—checked offline for pure instructions by a shipped bounded verification checker and bracketed at runtime by pre-dispatch context validation; (2) Agentic Calling, lifting provider-specific function calling into reusable, cacheable instructions; (3) instruction learning that mines recurring patterns from traces and promotes them as composite-replay instructions; and (4) a compiler and runtime applying classical optimization—fusion, dead-code elimination, pipeline scheduling, and dynamic hot-path specialization—to agent instruction streams.

We evaluate AgentISA on four fronts. A reproducible microbenchmark suite drives the real compiler, cache, and optimizer against a mock LLM backend: result caching reduces simulated API calls by 59.5–79.6% on synthetic Zipf workloads (2,000 tasks, 5 seeds), a saving that grows with skew; static compilation reduces the effective instruction count by 34.2% (35.4% at sequence length 1,000), with a same-seed ablation decomposing the shrink into fusion (21.6%) and dead-code elimination (24.5%), the two passes overlapping by 11.9 points (Section 5.5); dynamic hot-path optimization shrinks repeatedly executed traces by 65.8%. Two real-gateway experiments extend the validation beyond the mock: the cache reproduces the computed hit/miss statistics at every skew level and converts avoided calls into measured wall-clock and token savings, and the compiled, hot-path-adopted stream issues zero new gateway calls with byte-identical artifacts (cache-mediated). A four-task system-level characterization (12.5–25.0% per-task compile shrink) calibrates the synthetic headline, and 43 multi-agent coordination and security tests provide functional validation. We position these results as controlled-setting evidence, not as production end-to-end measurements. The shipped prototype implements result caching, the static passes that fire in the benchmarks (fusion, dead-code elimination, parallel grouping), and threshold-triggered trace re-optimization; tree-pattern instruction selection, cycle-accurate pipeline scheduling, and symbolic-equivalence checking remain specified components rather than fully implemented mechanisms, and capability metric extraction is a stub—no claim in this paper rests on them.

Keywords

- Instruction Set Architecture
- Large Language Models
- AI Agent Programming
- Compiler Optimization
- Multi-agent Systems
- Agentic Calling
- Trace-based Instruction Learning

- Capability-based Agent Coordination
- Formal Methods
- Result Caching

Research Highlights

1. A formal instruction set architecture that makes LLM agent behavior compilable, analyzable, and optimizable
2. Result caching reduces simulated API calls by 59.5–79.6% on synthetic Zipf workloads (5 seeds), with the reduction growing as workload skew increases
3. Static compilation (fusion + dead-code elimination) reduces the effective instruction count by 34.2% $\pm$ 1.8%, decomposed by a same-seed paired ablation into fusion (21.6%) and dead-code elimination (24.5%), with a -11.9-point overlap (Section 5.5)
4. Dynamic hot-path optimization shrinks repeatedly executed traces by 65.8% $\pm$ 4.6%
5. Agentic Calling lifts provider-specific function calling into reusable, provider-independent agent instructions
6. Real-gateway validations (Sections 5.7 and 5.7.1): observed cache hit rates match the computed expectation exactly, and the compiled, hot-path-adopted stream issues zero new gateway calls with byte-identical artifacts (cache-mediated)
7. A four-task system-level characterization calibrates the synthetic headline (12.5–25.0% per-task compile shrink, Section 5.8)

CCS Concepts

- **Software and its engineering** $\rightarrow$ **Compilers**: instruction selection, optimization, and scheduling applied to agent instruction streams
- **Theory of computation** $\rightarrow$ **Program reasoning**: type-safety and semantic-preservation arguments for instruction-level transformations (sanity results stated as design rationale)
- **Computing methodologies** $\rightarrow$ **Artificial intelligence**: LLM-based agents, tool use, and multi-agent coordination
- **Computer systems organization** $\rightarrow$ **Architectures**: instruction set design and runtime support for agent execution

Funding Information

No external funding was received for this work.

1 Introduction

Recent advances in Large Language Models (LLMs) have produced systems with strong natural language understanding, generation, and—through instruction tuning and human feedback—instruction-following capabilities [1, 2, 3, 4, 5, 6]. Building on these capabilities, a rapidly growing body of work couples LLMs with tools, memory, and control loops to construct autonomous agents [9, 10, 11, 12]. Yet the engineering substrate beneath these agents remains largely ad hoc: behaviors are specified as prompts and provider-specific function-calling schemas, which are difficult to specify precisely, analyze formally, or optimize systematically.

1.1 Background and Motivation

Function calling—selecting an API from a declared tool set and emitting structured arguments—has become the dominant interface between LLMs and external systems [13, 14, 15]. Standardization efforts such as the Model Context Protocol [18] are consolidating this interface layer, while evaluation harnesses such as the Berkeley Function Calling Leaderboard [17] measure it. However, the unit of interaction remains the individual API call: each call is dispatched, priced, and forgotten independently, even when an agent repeatedly executes structurally identical work.

This per-call statelessness stands in sharp contrast to how computing systems have historically handled repetitive execution. Classical compilers and hardware translate recurring patterns into cheaper forms—instruction fusion, dead-code elimination, caching, and trace-based dynamic optimization [33, 34, 45]. Our motivating hypothesis is that these same techniques apply to LLM agent workloads: if agent behavior is expressed as an explicit instruction stream rather than as opaque prompts, then recurring patterns become visible to a compiler and can be optimized with well-understood, semantics-preserving transformations.

1.2 Challenges in Current LLM-based Agent Systems

Five structural challenges motivate AgentISA:

1. **Per-call statelessness of tool interfaces.** Mainstream function-calling and tool-use protocols treat each call as an isolated event [13, 14, 18]. The provider has no notion of a *program* being executed, so repeated identical work incurs repeated cost; caching, when present, is a bolt-on afterthought rather than a semantic property of the execution model.

2. **Experience that does not accumulate as compiler-visible instruction structure.** Agent paradigms such as Reflexion [10] and Voyager [12] do accumulate experience, but as verbal reflections or skill libraries consumed by the model at inference time. The lessons are not reified as reusable, statically optimizable instructions that a compiler can fuse, schedule, or eliminate.

3. **Absence of a shared behavioral contract.** Each multi-agent framework defines its own abstractions for roles, messages, and workflows [19, 20, 21, 22, 23, 24]. There is no instruction-level contract that is independent of any single framework, which fragments tooling and makes cross-framework analysis difficult.

4. **Cost pressure at the wrong layer.** FrugalGPT [28] demonstrates that LLM inference cost is a first-order concern, and the serving-systems community has attacked this below the API boundary with paged attention, continuous batching, KV-cache-centric architectures, and attention kernels [38, 39, 40, 41]. What is missing is the complementary, above-the-API layer: reducing the *number and size* of calls an agent issues in the first place.

5. **Integration complexity across providers.** Portability across LLM providers is handled today primarily at the library level [26, 27]. Agent logic written against one provider's function-calling surface does not transfer cleanly to another, and capability differences between providers are hard to reason about systematically [17].

1.3 Research Objectives and Contributions

This paper introduces AgentISA, an instruction set architecture that addresses these challenges by making agent behavior a first-class compilable artifact. Our contributions are:

1. **A formal instruction set** for LLM agents with compositional semantics, a type system, and pre/post-conditions on every instruction (Section 3.1). Two deliberately modest sanity results are stated as design rationale rather than headline novelty: Turing completeness (Theorem 3.1), read as negative certification that the instruction set does not constrain expressiveness, and type safety for the pure core (Theorem 3.2), which fixes the well-formedness obligation that the offline bounded checker discharges for pure instructions (Section 3.3.3) and that the runtime brackets with pre-dispatch context validation.
2. **Agentic Calling**, a mechanism that lifts provider-specific function calling into reusable, cacheable, semantically specified agent instructions, with a model-adapter layer for cross-provider portability (Section 3.2).
3. **Trace-based instruction learning**, which mines recurring instruction patterns from interaction traces using frequency-based sequence mining and deterministic behavior grouping, and promotes validated recurring patterns into candidate instructions whose `execute` bodies replay the mined constituent instruction sequence by sequential composition over a shared context (Section 3.3; replay reproduces the recorded subsequence's outputs on the same inputs and does not generalize beyond them, as the section states).
4. **A compiler and runtime** that apply instruction selection, fusion, dead-code elimination, pipeline scheduling, and dynamic hot-path specialization to agent instruction streams (Sections 3.4 and 4), with semantic-preservation arguments for each transformation (Theorems 3.4–3.6, proofs in Appendix C).
5. **An empirical evaluation** of the implemented system, with full methodology disclosure. On synthetic Zipf-distributed workloads (5 random seeds each), caching reduces simulated API calls by 59.5–79.6% depending on skew, static compilation reduces the effective instruction count by 34.2% $\pm$ 1.8% (sequence length 200), and dynamic optimization shrinks repeatedly executed traces by 65.8% $\pm$ 4.6%. Two real-gateway experiments (Sections 5.7 and 5.7.1) validate the cache and the compiled pipeline against a real LLM gateway, and a four-task system-level characterization (Section 5.8) composes the mechanisms on realistic (non-synthetic) instruction streams.
6. **Multi-agent coordination infrastructure** with capability-based agent selection, execution planning, fallback strategies, and a security monitor enforcing authentication, authorization, resource-access, and data-sensitivity policies (Section 3.5 and Section 5.6), validated by 43 passing functional tests.

Our empirical claims are scoped accordingly: Sections 5.2–5.5 exercise the real `agentflow.core.isa` implementation against a mock LLM backend on synthetic workloads, and Sections 5.7–5.8 extend the validation at reduced scale on realistic streams. None of these are production end-to-end measurements (threats to validity in Section 5.9).

The rest of this paper is organized as follows: Section 2 reviews related work. Section 3 details the design of AgentISA. Section 4 describes the implementation. Section 5 presents the empirical evaluation. Section 6 discusses limitations and future work, and Section 7 concludes.

2 Related Work

AgentISA draws on eight lines of work: foundation LLMs and their function-calling interfaces, reasoning-and-acting agent paradigms, tool-calling standardization, multi-agent frameworks,

prompt compilation and cost optimization, agent benchmarks, classical compiler and architecture techniques, and LLM serving systems. We review each in turn and position our contributions.

2.1 Foundation LLMs and Function Calling

Modern LLMs are autoregressive transformers trained at large scale [1, 2, 3, 4], with instruction tuning and reinforcement learning from human feedback making them reliable instruction followers [5, 6]. Their text distribution is conventionally factorized as:

$$P(y|x) = \prod_{t=1}^{T} P(y_t|x, y_{<t}; \theta)$$

where x is the input context, y the generated output, and θ the model parameters. Function calling constrains this distribution to a structured output space:

$$F(x) = \text{argmax}_{f \in \mathcal{F}} P(f|x, \mathcal{F})$$

where $\mathcal{F}$ is the set of declared functions and $F(x)$ the selected call. GPT-4 [3], PaLM [2], and open models such as LLaMA [4] expose this capability with provider-specific schemas. AgentISA does not modify this layer; it treats any such interface as a *backend* to be targeted by a common instruction set, in the same way that a compiler targets multiple processor backends [33].

The cost structure of this layer motivates our work. If each call carries latency τ_i plus a context-switching overhead ω_i (prompt reconstruction, serialization, cache misses; the symbol δ is reused in Appendix B for per-instruction transition functions—the two are unrelated), then n similar calls cost:

$$C(n) = \sum_{i=1}^{n} (\tau_i + \omega_i)$$

Definition 2.1 (Function Call Overhead): The overhead of executing n function calls is $C(n)$, the sum of per-call latencies and per-call context reconstruction costs. When repeated calls carry identical semantic inputs and satisfy the cache-validity conditions of Section 3.2, the repeated context-construction cost ω_i is avoidable—which is precisely the cost AgentISA's caching and fusion are designed to remove. Structural similarity alone does not make a re-invocation waste: if an input's dependencies have changed or the call probes external state, re-execution has semantic meaning, and the cache-validity conditions—not structural identity—are what license reuse.

2.2 Reasoning and Agent Paradigms

A second line of work enhances LLM reasoning at prompt time. Chain-of-thought prompting [7] and zero-shot reasoning [8] improve multi-step behavior by eliciting intermediate steps. ReAct interleaves reasoning traces with actions [9]; Reflexion adds verbal self-critique across episodes [10]; Tree of Thoughts explores deliberative search over thought trees [11]; and Voyager demonstrates open-ended skill acquisition in an embodied environment [12]. These paradigms define *what* an agent should do at the cognitive level. AgentISA is complementary: it defines *how* the resulting behaviors are represented, compiled, and executed as instructions. A ReAct loop, for instance, can

be expressed as an AgentISA instruction sequence whose tool invocations become fused, cacheable agentic calls.

2.3 Tool Calling and Interface Standardization

Toolformer trains models to interleave tool calls with text [13]; Gorilla connects models to large API corpora [14]; ToolLLM scales tool coverage to 16,000+ real-world APIs [15]; and HuggingGPT orchestrates model hubs for task solving [16]. On the standardization side, the Model Context Protocol defines an open specification for exposing tools and context to LLM applications [18], and the Berkeley Function Calling Leaderboard provides a systematic cross-model evaluation of function-calling accuracy [17].

These efforts standardize the *interface*. AgentISA standardizes the *execution model above the interface*: it compiles a provider-independent instruction stream down to whatever interface a backend exposes. This distinction matters for portability—an AgentISA program compiled for two providers differs only in the final translation step (Section 3.2.3)—and for optimization, because interface-level standards alone cannot fuse or eliminate calls.

2.4 Multi-Agent Frameworks

AutoGen frames agent applications as multi-agent conversations [19]; MetaGPT assigns standardized roles and workflows to agents [20]; AgentVerse facilitates multi-agent collaboration and studies emergent behaviors [21]; ChatDev organizes agents as a software-development organization [22]; CAMEL studies communicative agent societies [23]; and AutoGPT popularized autonomous goal-driven agents [24].

AgentISA contributes a different artifact to this space: not a framework or orchestration philosophy, but an instruction-level substrate—capability profiles, coordination instructions with barrier semantics, and a security monitor—that any such framework could target. Concretely, our multi-agent layer (Section 3.5, Section 5.6) provides capability-based agent selection, execution planning with per-stage fallback strategies, and policy enforcement for authentication, authorization, resource access, and data sensitivity.

2.5 Prompt Compilation and Cost Optimization

Closest to our compilation perspective is DSPy, which compiles declarative prompt programs into optimized pipelines via prompt search and fine-tuning [25]. LangChain [26] and LlamaIndex [27] provide library-level abstractions for chaining LLM calls and indexing context. FrugalGPT reduces cost through model cascades—answering with cheaper models when possible [28].

AgentISA differs in the level at which optimization operates. DSPy optimizes *prompts*; FrugalGPT optimizes *model selection*; AgentISA optimizes *instruction streams*: instruction selection by dynamic programming over pattern covers, fusion, dead-code elimination, pipeline scheduling, result caching, and dynamic hot-path specialization (Sections 3.4 and 4). These are classical compiler transformations [33] applied to a new artifact, and they compose with—rather than replace—prompt-level and model-level optimization.

2.6 Benchmarks for Agent Systems

AgentBench evaluates LLMs as agents across diverse environments [29]; GAIA tests general assistant abilities [30]; PlanBench probes planning and reasoning about change [31]; and SWE-bench

measures resolution of real-world GitHub issues [32]. These benchmarks evaluate *end-to-end capability*. AgentISA's evaluation instead measures *subsystem behavior under controlled conditions*—API-call reduction under synthetic Zipf workloads, instruction-count reduction from compilation, and trace shrinkage from dynamic optimization (Section 5). We view the two as complementary: instruction-level metrics can attribute end-to-end gains to specific mechanisms when such gains are measured, and are reproducible without live model access.

2.7 Classical Compiler and Architecture Techniques

AgentISA's optimization machinery is deliberately classical. Instruction selection by tree-pattern matching with dynamic programming, peephole optimization, and data-flow-based dead-code elimination are standard compiler techniques [33]. Pipelining, hazard detection, and resource-constrained scheduling follow the classical architecture literature [34]; the RISC-V project demonstrates the value of open, well-specified instruction sets [35]. Type safety via progress-and-preservation is the standard technique of programming-language theory [36], and CompCert establishes that compiler correctness itself can be formally verified [37]. Our contribution is not a new compiler technique but the demonstration that this machinery transfers to LLM agent workloads, together with the semantic-preservation arguments adapted to instructions with pre/post-conditions (Appendix C).

2.8 LLM Serving Systems

Below the API boundary, the serving community has made inference dramatically cheaper: PagedAttention manages KV memory like virtual memory [38]; Orca introduces continuous batching for transformer serving [39]; FlashAttention redesigns attention for IO-awareness [40]; and Mooncake proposes a KVCache-centric disaggregated architecture [41]. Energy analyses quantify the inference cost of large models [42]. This line of work and AgentISA are orthogonal layers of the same stack: serving systems make each call cheaper; AgentISA reduces how many calls an agent makes and how much redundant work each carries. The two compound multiplicatively.

2.9 Research Gaps and Our Contributions

In summary, the literature leaves four gaps that AgentISA addresses:

1. **No instruction-level contract for agent behavior.** Interfaces are standardized [13, 14, 18], and frameworks are numerous [19, 20, 21, 22, 23, 24], but no shared instruction set spans providers and frameworks.
2. **No compiler for agent behavior.** Prompt-level compilation exists [25] and model-level cost reduction exists [28], and *caching itself is well-trodden ground*—at the response level (GPTCache [46]), at the KV-prefix level (RadixAttention/SGLang [47]), and at the provider prompt-caching level. What none of these layers expose is an *instruction-level* contract: caching there is an optimization of model serving—invisible to the agent program's structure, and without effect on it. Instruction-level fusion, elimination, and scheduling—with cacheability as a declared property of the instruction set—have not, to our knowledge, been demonstrated.
3. **No semantics-preserving account of agent optimization.** Serving work verifies kernels and schedulers, not agent-level transformations; we provide progress-and-preservation-style arguments for ours (Theorems 3.2, 3.4, 3.5, 3.6).

4. **No reproducible instruction-level evaluation.** Existing benchmarks measure end-to-end capability [29, 30, 31, 32]; we contribute a seed-controlled evaluation: a microbenchmark suite with fully disclosed synthetic workloads and mock-backend instrumentation, extended by real-gateway and system-level validations (Section 5).

AgentISA addresses these four gaps directly—the instruction set and Agentic Calling abstraction (Sections 3.1–3.2), instruction-level optimization with semantic-preservation arguments (Section 3.4, Theorems 3.2 and 3.4–3.6, Appendices B–C), trace-derived instruction learning (Section 3.3), and a seed-controlled evaluation (Sections 5.2–5.8); the full contribution list is in Section 1 and is not repeated here.

3 AgentISA Design

Building upon the foundations established in Sections 1 and 2, we present AgentISA, a comprehensive instruction set architecture designed specifically for LLM-powered AI agents. This section details the core architecture, the Agentic Calling framework, the automatic instruction learning system, instruction organization and optimization, and the capability-assessment layer.

3.1 Core Architecture

3.1.1 Instruction Set Specification

The AgentISA instruction set is formally defined as a tuple:

Definition 3.1 (AgentISA Core):

$$\text{AgentISA} = (I, T, \Sigma, \Delta, \Phi)$$

where:

- I is the set of instructions
- T is the type system
- Σ is the state space
- Δ is the transition function set
- Φ is the verification function set

Following the conventions of classical instruction set design [33, 34, 35], we define the basic instruction format:

```
INSTRUCTION = {
    opcode: OPCODE,
    args: ARGS[],
    precondition: PRED,
    postcondition: PRED,
    context: CONTEXT
}
```

The instruction set is categorized into four fundamental classes:

1. **Control Flow Instructions**:

$$\mathcal{C} = \{seq, branch, loop, call\}$$

2. **State Management Instructions**:

$$\mathcal{S} = \{load, store, update, cache\}$$

3. **LLM Interaction Instructions**:

$$\mathcal{L} = \{query, response, context\}$$

4. **Resource Management Instructions**:

$$\mathcal{R} = \{allocate, free, monitor\}$$

Two sanity results anchor the instruction-set definition. Neither introduces new machinery—the simulation technique from the compiler literature [33], the type-safety technique from the semantics literature [36]—and we state them for the roles they play in the architecture: Theorem 3.1 certifies that the instruction set does not artificially constrain what agent programs can express, and Theorem 3.2 fixes the well-formedness obligation that the offline bounded checker discharges for pure instructions (Section 3.3.3) and that the runtime brackets with pre-dispatch context validation (Section 3.2.3).

Theorem 3.1 (Turing Completeness): The AgentISA instruction set is Turing complete.

Proof sketch: We exhibit a translation from WHILE programs—already known to be Turing equivalent—to AgentISA instruction sequences, using a sufficiency (simulation) argument over instruction classes: state management instructions encode an unbounded store, and control-flow instructions encode sequential composition, conditional branching, and iteration. The full construction and simulation argument are given in Appendix A, following standard simulation techniques from the compiler literature [33] and the operational-semantics framework of Pierce [36].

3.1.2 Type System and Semantics

The type system T extends the simply-typed discipline of Pierce [36] with dependent-type constructions (Π, Σ, μ):

Definition 3.2 (AgentISA Type System):

$$T ::= \text{Base} \mid \Pi(x : T_1).T_2 \mid \Sigma(x : T_1).T_2 \mid \mu X.T$$

where:

- Base types include {Bool, Int, String, Float}
- Π represents dependent function types
- Σ represents dependent pair types
- μ represents recursive types

The semantic rules follow operational semantics:

$$\frac{e_1 \to e_1'}{E[e_1] \to E[e_1']} \text{ (Context)} \quad \frac{}{(\lambda x.e)v \to e[x := v]} \text{ (β-reduction)}$$

The pure core carries a standard type-safety argument—progress and preservation, in the style of Pierce [36]—discharging the well-formedness role fixed in Section 3.1 and extended to the shipped instruction set by the bounded checker of Section 3.3.3:

Theorem 3.2 (Type Safety for the Pure Core): Well-typed programs in the pure core calculus of Appendix B—the λ-fragment (base types, dependent Π/Σ types, recursion) plus the composition operators applied to instructions carrying typed pre/post-conditions—cannot "go wrong", under the per-instruction validity obligation of Definition 3.5 (stated in Section 3.3.3 below; pre/post-conditions), with transition functions assumed total on precondition-satisfying states (Appendix B.7). Effectful instruction classes are embedded via their typed pre/post-conditions and covered to the extent their preconditions are checkable.

Proof sketch: By the standard two-lemma technique—progress and preservation—as presented in Pierce [36]:

1. Progress: $\forall e, T.\ \vdash e : T \implies \text{value}(e) \vee \exists e'.\ e \to e'$
2. Preservation: $\forall e, e', T.\ \vdash e : T \wedge e \to e' \implies \vdash e' : T$

Both lemmas are proved by induction over the derivations of the merged judgment system of Appendix B (term typing plus contracted instruction typing), with a joint argument for the composition cases. For instruction steps, progress follows from the Definition 3.5 validity obligation plus the standing totality assumption, and preservation from the postcondition typing; composition operators are handled by the chained invariant $post(i_1) \Rightarrow pre(i_2)$ (Appendix B).

3.1.3 LLM Integration Interface

The LLM integration interface is defined through a protocol, playing the same role for agent backends that the Model Context Protocol plays for tool exposure [18]. Its components correspond to the interface conventions of modern function-calling providers [3, 18] as follows—$\mathcal{P}_{LLM} = (Q, R, C, \psi)$ with $Q \leftrightarrow$ QueryFormat (query space, JSON Schema), $R \leftrightarrow$ ResponseFormat (response space, JSON Schema), C the context-management functions (whose implementation surface today exposes the context window size, ContextWindow: Integer), and $\psi \leftrightarrow$ StateMapping: Map<AgentState, LLMContext> (state transformation functions):

```
Protocol LLMInterface {
    QueryFormat: JSON Schema
    ResponseFormat: JSON Schema
    ContextWindow: Integer
    StateMapping: Map<AgentState, LLMContext>
}
```

3.2 Agentic Calling Framework

3.2.1 Function Calling to Agentic Calling Transformation

The transformation from function calling to Agentic Calling is formalized as:

Definition 3.3 (Agentic Transformation):

$$\mathcal{T} : F \to A$$

where F is the function call space and A is the agent instruction space.

$\mathcal{T}$ is required to preserve call semantics, and this holds by construction for the three-step process below: the synthesized instruction records the function signature verbatim, embeds the context verbatim (Theorem 3.3), and delegates execution to the original call. We therefore state preservation as a property of the construction rather than a separately proved theorem:

$$\forall f \in F : \text{sem}(\mathcal{T}(f)) \equiv \text{sem}(f)$$

The transformation process follows three steps, mirroring the signature-to-call conventions of function-calling interfaces [13, 14]:

1. Function signature analysis
2. Context embedding
3. Instruction synthesis

3.2.2 Context Preservation and State Management

State management in AgentISA follows a formal state machine model:

$$M = (S, s_0, \delta, F)$$

where:

- S: Set of states
- s_0: Initial state
- δ: State transition function
- F: Set of final states

Extraction-time context preservation is guaranteed by construction (Theorem 3.3); at runtime it is only monitored:

Theorem 3.3 (Context Preservation): Let ϕ be the context interpretation function (defined compositionally over context *contents*—the keys and values the context carries), and t the extraction transformation that embeds a context c verbatim into an instruction's `context` field. Then, for every $c \in C$:

$$\phi(t(c)) = \phi(c)$$

Proof: By the definition of t, the extracted instruction carries a verbatim copy of the source context; since ϕ is defined over context contents and the copy is content-identical, $\phi(t(c)) = \phi(c)$ holds by construction. This is a *design invariant of the extraction pipeline*: it holds because the implementation copies rather than re-encodes. At runtime the shipped checks are constraint-based and bracket the transition: `AgenticTransformer.validate_context` checks the registered context constraints pre-dispatch, and `verify_context_preservation(before, after)` re-checks them pairwise across the executed transition and reports violations (mechanics in Section 3.2.3); neither performs structural validation of the carried context. Theorem 3.3's extraction-time preservation is discharged by the copy construction; preservation across an executed transition is *monitored, not guaranteed*, by the runtime checks. ■

3.2.3 Cross-model Compatibility

Cross-model *protocol* compatibility—not behavioral equivalence of models—is achieved through an abstract interface layer (provider-independent instruction representation with provider-specific adapters):

```
Interface ModelAdapter {
   translate: AgentInstruction → ModelSpecificCall     // shipped: translate_instruction
   normalize: ModelSpecificResponse → AgentResponse    // shipped: normalize_response
   validateContext: Context → Boolean                  // shipped: AgenticTransformer.validate_context
}
```

All three interface operations ship. `translate` is `ModelAdapter.translate_instruction` (with registered type transformations). `normalize` is `ModelAdapter.normalize_response`, which folds the two dominant wire shapes—the Anthropic protocol (typed `content` blocks with `usage.input_tokens`/`output_tokens`) and the OpenAI protocol (`choices[0].message` with `usage.prompt_tokens`/`completion_tokens`)—into a canonical `AgentResponse` envelope (content, tool calls, usage, model, raw), so downstream instruction code never branches on provider syntax. `validateContext` is `AgenticTransformer.validate_context`, the pre-execution complement of the shipped post-hoc `verify_context_preservation(before, after)`: it evaluates the registered context constraints against a single state before dispatch, applying two-argument pairwise predicates reflexively, (v, v) (single-argument predicates apply directly), and leaving constraints over absent keys vacuously satisfied in line with the preservation semantics—so key *deletion* goes unreported, a blind spot analogous to the non-reflexive constraints below. Constraints that are non-reflexive by design, e.g. strict growth, are not detectable pre-execution.

Motivated by the observation of cross-model function-calling studies [17] and open protocol efforts [18]—providers differ in surface syntax but can agree on *protocol shape*—the requirement is a **protocol-level contract on the adapter layer**, not a claim of behavioral equivalence between models. For $\mathbb{M}$ the set of provider adapters, each $m \in \mathbb{M}$ provides a request normalizer $\text{norm}_{in}(m)$ and a response normalizer $\text{norm}_{out}(m)$ such that:

$$\forall m \in \mathbb{M} : \text{norm}_{in}(m), \text{norm}_{out}(m) \text{ are total on the supported request and response classes,}$$

mapping every supported raw request into the unified instruction-side schema and every supported raw response into the unified result schema, with declared types preserved and errors classified into a common error taxonomy—so that downstream instruction code never branches on provider syntax. This is deliberately weaker than a semantic-output equivalence: nothing here implies that two models given the same normalized input return the same semantic content (taking $m_1 = m_2$ would already demand determinism, which no LLM provides), and cross-model *behavioral* comparison is an empirical question at benchmark level (Section 3.5, [17, 29]), not something an interface definition can deliver.

3.3 Trace-Based Instruction Learning

3.3.1 Dialogue Pattern Recognition

Pattern recognition in AgentISA is frequency-based and deterministic (action-signature grouping) rather than grammatical; it is implemented in `agentflow.core.isa.pattern_mining` and `agent-`

`flow.core.isa.instruction_learning`. To prevent over-reading: the subsystem is a **discovery-and-replay pipeline, not an autonomous learning loop**. Its five stages are (i) trace-derived pattern mining (this section), (ii) dialogue-derived template extraction (§3.3.2—templates carry no recorded instruction body and raise `NotImplementedError` if executed), (iii) candidate validation (support and success-rate thresholds), (iv) *developer-controlled* promotion (mining never edits the optimizer's rule set; a human decides which candidates become instructions), and (v) composite replay (a promoted instruction re-runs its recorded subsequence on the same inputs; it does not generalize to unseen parameter values). Stages (i)–(iii) and (v) ship; stage (iv) is deliberately manual.

Definition 3.4 (Instruction Sequence Pattern): A pattern p over the instruction alphabet I is a non-empty contiguous sequence $p = (i_1, \ldots, i_k) \in I^k$ together with two frequency statistics:

- **Support**: $sup(p) = P(p \text{ occurs as a contiguous subsequence in a randomly drawn trace})$
- **Confidence**: $conf(p \Rightarrow j) = P(i_{t+1} = j \mid (i_{t-k+1}, \ldots, i_t) = p)$

A pattern is *frequent* if $sup(p) \geq \sigma_{min}$ and *predictive* if $\max_j conf(p \Rightarrow j) \geq \gamma_{min}$, for thresholds $\sigma_{min}, \gamma_{min} \in (0, 1]$.

Pattern recognition proceeds in three stages:

1. **Frequent-sequence mining.** Candidate subsequences are enumerated from execution traces together with their occurrence counts; successor statistics are accumulated alongside, and sequences below σ_{min} are pruned together with their statistics—so confidence is estimated only over frequent sequences. The output is the set of frequent, predictive patterns.

2. **Behavior grouping.** The shipped `PatternRecognizer` groups dialogues deterministically by *action signature*—the sorted set of system-turn contents—and promotes each signature bucket with sufficient support into a pattern—and, for dialogue-derived buckets, an acceptable success rate. An earlier prototype used TF-IDF embeddings with density-based clustering (DBSCAN) and k-means over instruction-trace feature vectors; we abandoned it for production use because it was non-deterministic across library versions and degenerate on the small, short samples that agent traces actually provide. (A k-means view over instruction-trace features remains available in `pattern_mining`, with fixed `random_state` for reproducibility.) Each signature bucket groups same-kinded behaviors by construction (identical action *sets*; argument values and turn counts may differ). Promotions split by source: trace-mined promotions execute by **composite replay** rather than as placeholders—the mined pattern carries its constituent instruction sequence together with the dataflow edges between its members, and the learned instruction's `execute` body re-runs that sequence in mined order against a shared context built from its inputs, merging each constituent's outputs and returning the keys declared by its `produces_X` postconditions; dialogue-mined templates, which carry no recorded instruction body, raise `NotImplementedError` rather than silently no-opping. Promotion is deterministic: pattern identity is derived from the mined instruction-id sequence, so the same traces promote the same instructions, in the same order, across runs. Replay semantics are plain sequential composition—nothing smarter—so a learned instruction reproduces the outputs of its recorded subsequence on the same inputs but does not generalize to unseen parameter values; no automated mining-to-optimization decision path exists yet (promotion is a developer decision).

3. **Pattern validation.** Each candidate pattern is checked for significance—support above σ_{min} (the frequency threshold of stage 1) and, for dialogue-derived patterns, an acceptable success

rate among the dialogues exhibiting it—before it is promoted to a candidate instruction.

This design is deliberately statistical: agent traces are short and noisy, and frequency statistics degrade gracefully where grammar induction would overfit. It also composes with the compiler—a frequent, verified pattern is exactly the candidate for promotion to a fused instruction (Section 3.4.4).

3.3.2 Instruction Extraction Pipeline

The instruction extraction process is the three-stage pipeline of Section 3.3.1, instantiated as shipped (stage names from Section 3.3.1 in parentheses):

1. **Pattern Detection** (frequent-sequence mining): frequent-sequence mining over execution traces, accumulating support and confidence statistics; sequences below the support threshold are pruned.
2. **Instruction Formation** (behavior grouping): deterministic action-signature grouping promotes validated patterns into composite-replay candidates; the extracted template carries the recorded constituent sequence and its dataflow edges.
3. **Optimization and Validation** (pattern validation and optimization): threshold and success-rate checks gate promotion, and the standard optimization passes of Section 3.4 apply to any resulting stream.

3.3.3 Instruction Optimization and Verification

Optimization is coupled to verification: where verified-compilation efforts such as [37] verify the transformations of a whole compiler, AgentISA's verification targets a different granularity—the validity of individual instructions and the obligations of individual rewrites:

Definition 3.5 (Instruction Verification): An instruction i is valid if:

$$\forall s \in S : \mathrm{pre}(i, s) \implies \mathrm{post}(i, \mathrm{exec}(i, s))$$

For pure instructions with finite, small state spaces, the shipped checker (`agentflow.core.isa.bounded_checker`) enumerates the candidate input states derived from the declared parameter constraints—exhaustively for finite domains such as `bool` and enum parameters—and checks the implication on each state; for larger or non-enumerable spaces it probes a bounded, deterministically sampled set of states (default cap 256), which makes it a *bounded* verification procedure rather than a complete one. We state the implementation status plainly: Definition 3.5 defines validity, and the bounded checker is now **shipped code**: for every pure (side-effect-free) instruction the checker generates the bounded state space, filters it by well-typedness and by the declared preconditions, executes the instruction, and evaluates three obligations on each surviving state—progress (execution must not raise where the precondition holds), preservation (declared outputs must be present and well-typed), and the postcondition itself. Side-effectful instructions are never executed by the checker and are reported as out of scope, so their validity obligation is not mechanically checked; it is delegated to authoring-time unit tests and review. Theorem 3.2 is stated conditionally on this obligation, and Theorems 3.4–3.6 carry the separate rewrite obligations of Appendix C.7; for state spaces beyond the enumeration bound the condition remains substantive rather than discharged (Appendices B.7 and C.7):

$$\forall s \in S_{bounded} \subseteq S : \text{pre}(i, s) \implies \text{post}(i, \text{exec}(i, s))$$

where $S_{bounded}$ is the explored subset. Completeness of the check is traded for tractability—the standard bounded-exploration compromise; the operational-semantics treatment of instruction meaning itself follows [36].

The guarantee therefore comes in two explicitly separate layers. **Layer 1 (theory, conditional):** Theorem 3.2 and the type-safety results of Appendix B hold for every program whose instructions satisfy Definition 3.5 *on all of* S, under the total-definedness premise of Appendix B.7—this is a mathematical statement about a proof obligation, not about the checker. **Layer 2 (implemented verification):** the shipped checker discharges that obligation only on $S_{bounded}$ (exhaustive enumeration for small finite domains; deterministic bounded sampling with cap 256 otherwise) and only for pure instructions; side-effectful instructions are reported out of scope and their obligation is delegated to unit tests and review. Success of the bounded check is evidence toward, not a proof of, Layer 1's premise; no claim in this paper equates the two.

Theorem 3.4 (Optimization Soundness): Every atomic rewrite of the pass pipeline—fusion (Appendix C.2's rule, applied by the combining pass), dead-code elimination, and instruction combining (Section 3.4.4)—preserves the denotation of the instruction sequence it transforms: $\text{sem}(r(i)) \equiv \text{sem}(i)$, read as the partial equivalence of C.1.

Proof sketch: By structural induction on instruction composition. The base case shows each atomic rewrite (fusion, elimination, combining) preserves the denotation of a single instruction; the inductive step lifts preservation through each composition operator of Definition 3.6. The full proof, including the per-rewrite case analysis, is given in Appendix C.

The shipped checker (`agentflow.core.isa.bounded_checker.verify_instruction`) implements Definition 3.5 on the bounded state space exactly as described above (enumerate, filter by preconditions, execute only pure instructions); it reports violations structurally:

```
def verify_instruction(instruction, pre_condition, post_condition):
    if instruction.side_effects: return skipped(instruction)  # never executed
    state_space = generate_state_space(instruction)  # bounded: exhaustive or seeded, cap 256
    for state in state_space.states:
        if pre_condition(state):
            next_state = execute(instruction, state)
            check(progress, preservation, post_condition, next_state)  # structured violation re
    return verdict  # checked_states, exhaustive, violations by kind, passed
```

3.4 Instruction Organization and Optimization

3.4.1 Instruction Composition

The AgentISA supports hierarchical instruction composition through a formal composition algebra:

Definition 3.6 (Instruction Composition): Given instructions $i_1, i_2 \in I$, we define composition operators:

$$\text{Sequential} : i_1 \circ i_2$$
$$\text{Parallel} : i_1 \parallel i_2$$
$$\text{Conditional} : i_1 \rhd i_2$$
$$\text{Iterative} : i_1^*$$

Their denotations (sequential execution; parallel interleaving under disjoint state footprints; guarded execution of i_2 on i_1's triggering outcome; zero-or-more iterations of i_1) are fixed in Appendix C.1.

Composition follows algebraic properties. Properties 1 and 3 hold for all instructions; Property 2 is a *conditional* law—purity and pairwise footprint disjointness together do *not* suffice, and idempotence of i_1 must be assumed separately:

1. **Associativity** (all instructions, up to denotation):

$$(i_1 \circ i_2) \circ i_3 = i_1 \circ (i_2 \circ i_3)$$

2. **Distributivity** (conditional on idempotent i_1 and pairwise-disjoint footprints): the law

$$i_1 \parallel (i_2 \circ i_3) = (i_1 \parallel i_2) \circ (i_1 \parallel i_3)$$

   holds under two premises that must be stated separately. **(a) Idempotence of** i_1: $\delta_{i_1} \circ \delta_{i_1} = \delta_{i_1}$ on the precondition-satisfying states—i_1's writes must not change the values its own subsequent execution reads (a constant overwrite of its declared outputs qualifies; a read-modify-write of a cell it also reads does not). **(b) Pairwise footprint disjointness**: the denotations of i_1, i_2, i_3 act on pairwise disjoint state components (declared outputs distinct, read sets disjoint from the other members' writes). Under (a), the second execution of i_1 on the right-hand side reproduces the first's writes; under (b), i_2 and i_3 cannot perturb i_1's inputs between its two occurrences; disjoint updates commute, so both sides denote the same net state transformation.

   Both premises are necessary, and neither follows from the other or from purity. That purity does not imply idempotence is the sharper failure: a pure `update(+1)` instruction applied twice increments twice (pure means free of *external* side effects; writes to declared state cells are permitted). Taking i_1 as such an update and $i_2 = i_3 = \epsilon$ (the identity, with an empty footprint, so premise (b) holds vacuously), the left side executes i_1 once and the right side twice: from $x = 0$ the two sides yield $x = 1$ and $x = 2$. The law therefore fails for a pure instruction whose footprints are disjoint from every other participant's—only premise (a) rules this out. Conversely, without (b), a write of i_2 into a cell i_1 reads changes i_1's second execution even when i_1 is idempotent.

3. **Identity**:

$$i \circ \epsilon = \epsilon \circ i = i$$

where ϵ is the empty (identity) instruction.

Remark (implementation consequence): distributivity also fails for side-effectful instructions, for the same double-execution reason. Because the law is doubly conditional, the shipped compiler does

not rely on it at all: parallel grouping is gated on purity plus disjoint dependencies and disjoint outputs, `side_effects=True` instructions are excluded from all such rewrites, and no distributivity-based simplification is applied. The law is stated for the algebra's completeness, not as a license for rewrites.

3.4.2 Instruction Pipeline Design

We formalize the instruction pipeline as a directed acyclic graph (DAG):

Definition 3.7 (Pipeline DAG):

$$P = (V, E, \lambda, \delta)$$

where:

- V: Pipeline stages
- E: Dependencies between stages
- λ: Stage latency function
- δ: Data transfer function

Pipeline stages are organized as:

```
Pipeline {
    FETCH → DECODE → SCHEDULE → EXECUTE → COMMIT
    with
        hazard_detection: HazardGraph
        dependency_tracking: DependencyMatrix
        resource_allocation: ResourceMap
}
```

Theorem 3.5 (Pipeline Correctness): For any valid instruction sequence s and any schedule S satisfying conditions S1-S4 of Appendix C.5—each instruction dispatched exactly once, dependencies respected, dispatch groups pairwise non-interfering, commit in program order—pipeline execution preserves program semantics.

Proof: By induction over the dispatch groups of S in commit order, under the invariant that the architectural state after committing each prefix of groups equals the sequential state after the same instructions; S1–S4 supply exactly-once dispatch, dependency order, group commutativity, and in-order commit (details in Appendix C.5).

3.4.3 Instruction Selection and Scheduling

Instruction selection follows a tree-pattern matching approach:

Definition 3.8 (Selection Pattern):

$$\pi = (\tau, C, \text{cost}, \text{code})$$

where:

- τ: a tree pattern over the IR (unrelated to the type-system symbol T of Definition 3.2)
- C: context constraints on the surrounding state
- cost: Cost function

- code: Generated code

Given an IR tree t to cover, the selection process is formalized as:

$$\text{Select}(t) = \arg\min_{\pi \in \Pi} \{\text{cost}(\pi) \mid \text{matches}(\pi.\tau, t) \wedge \text{satisfies}(t, \pi.C)\}$$

Scheduling optimization problem (with $\text{delay}(i)$ the incurred stall of i, $\text{active}(t)$ the instructions running at time t, $\text{res}(i)$ the resource demand of i, and Capacity the resource bound):

$$\begin{aligned}
&\text{minimize} \sum_{i \in I} \text{delay}(i) \\
&\text{subject to:} \\
&\quad \forall (i_1, i_2) \in \text{Deps} : \text{start}(i_2) \geq \text{end}(i_1) \\
&\quad \forall t : \sum_{i \in \text{active}(t)} \text{res}(i) \leq \text{Capacity}
\end{aligned}$$

3.4.4 Static and Dynamic Optimization

Static optimization. Static optimization employs several techniques:

1. **Peephole Optimization**:

$$\text{PH}(i_1, ..., i_n) = \begin{cases} \text{opt}(i_1, ..., i_n) & \text{if pattern exists} \\ i_1, ..., i_n & \text{otherwise} \end{cases}$$

2. **Dead Code Elimination**:

$$\text{Live}(i) = \text{Use}(i) \cup (\text{Live}_{\text{out}}(i) - \text{Def}(i))$$

   The two shipped passes approximate this fixpoint at different conservatism levels. The *compiler* pass (formal streams) removes a side-effect-free instruction when no live consumer depends on its id or any declared output—so a pure instruction whose outputs nobody consumes is removed. The *trace optimizer* pass (executable instructions) is more conservative: any instruction declaring at least one output is kept, because executable traces lack the formal dependency annotations that make output-consumption analysis exact. The compiler experiments (Section 5.3) exercise the former; the dynamic hot-path machinery (Section 5.4) the latter.

3. **Instruction Combining** (the named candidate rules below are specified and not registered—Section 4.1.3; the shipped optimizer's built-in pairwise compatibility combining is active, and it is this built-in combining that the hot-path trace re-optimization of Section 5.4 applies):

```
Combine Rules {
  // LOAD/STORE/MOVE are textbook mnemonics, illustrative
  // only; they are not shipped opcode names
  LOAD followed by STORE → MOVE
  Multiple CONTEXT_UPDATE → Single UPDATE
  Sequential QUERY → Batch QUERY
}
```

Dynamic optimization. Dynamic optimization in the shipped runtime is **repetition-triggered and threshold-based**; it does not (yet) respond to load or performance signals. Its actual mechanism:

1. **Hot Path Detection** (implemented): the `DynamicOptimizer` keeps a monotone counter per exact instruction sequence (`path_id` = the ordered join of instruction ids); a path is flagged hot when its count exceeds a configurable threshold *and* its most recent recorded success rate is at least a stability bound (0.9 by default). There is no windowing and no load signal:

$$\text{Hot}(p) = \text{count}(p) > \theta \wedge \text{success}(p) \geq \alpha$$

2. **Trace Optimization** (implemented): once hot, the cached sequence is re-optimized by the static pass machinery—peephole, dead-code elimination, instruction combining—bounded to a fixed number of passes; the runtime then serves the shrunken form for subsequent executions (demonstrated end-to-end in Section 5.8).

3. **Trace Formation and Adaptive Recompilation** (specified, not implemented): sub-trace extraction from observed frequency profiles, and performance-triggered recompilation (`should_recompile`: execution count above threshold *and* observed performance below expectation) remain design-level. The shipped runtime re-optimizes the whole cached sequence once per recording after the threshold is crossed, with no performance feedback loop.

Two known consequences of the shipped design are worth stating: because `path_id` requires exact sequence identity, trace diversity (similar-but-not-identical sequences) never accumulates counts and the mechanism stays cold; and hot-path state (sequences, counters, statistics) is unbounded. Both are discussed with their remedies in Section 6.1.

3.4.5 Verification and Analysis

Optimization correctness is argued through:

Theorem 3.6 (Pass-Pipeline Soundness, extending Theorem 3.4): For any optimization transform T composed of the pass pipeline's rewrites,

$$\forall p \in \text{Program} : \text{sem}(T(p)) \equiv \text{sem}(p)$$

Proof Sketch:

1. Show semantic preservation for each atomic transformation (the atomic-rewrite lemma, Appendix C.2)
2. Prove composition of transformations preserves semantics (the compositionality lemma, Appendix C.3)
3. Note that hazard-free pipeline execution is *not* part of this theorem—it is Theorem 3.5 (Appendix C.5), invoked separately; this theorem's contribution is the rewrite layer (steps 1–2).

Performance Model (specified; the benchmarks of Section 5 measure latency and token counts directly and do not evaluate it):

$$\text{Perf}(p) = \sum_{i \in p} (\text{base_cost}(i) + \text{overhead}(i) - \text{optimization_gain}(i))$$

where optimization_gain is calculated as:

$$\text{optimization_gain}(i) = \sum_{o \in \text{Opts}} \text{benefit}(o, i) \times \text{application_rate}(o, i)$$

3.5 Capability Assessment

Agents are heterogeneous: an agent (or the model behind it) may be strong at reasoning but weak at tool use, or vice versa. To make such differences explicit—and to give the multi-agent coordinator of Section 5.6 a principled basis for agent selection—AgentISA includes a capability assessment subsystem, implemented as `CapabilityProfile` in `agentflow.core.isa.capability`.

Definition 3.9 (Capability Profile): A capability profile of an agent a maps each capability domain D (the shipped enum carries eight domains: reasoning, memory, generation, optimization, specialization, coordination, security, robustness) to a score–weight pair:

$$\text{Cap}(a) = \{(\text{dom}_k, s_k, w_k) \mid \text{dom}_k \in D\}$$

where each domain carries a score $s_k \in [0, 1]$ and a weight $w_k \geq 0$ reflecting the domain's importance in the current deployment context.

Implementation status. The aggregation and selection mechanisms ship (`CapabilityProfile`, `get_overall_score`, and the coordinator's threshold selection) and are functionally tested in Section 5.6; the instruction-level metric extraction is a specified stub—the seventeen `_measure_*` functions currently return 0.0—so `assess_capabilities` has no production call site yet, and the tests of Section 5.6 inject `DomainScore` values directly rather than deriving them through this pipeline. The pipeline's *mechanics* are pinned by unit tests (`tests/core/isa/test_capability_pipeline.py`): end-to-end assessment over a real instruction list produces well-formed results for all eight domains with stub-zero scores, the domain aggregation is the confidence-weighted mean with the weight deliberately absent (a regression test for the double-weighting defect fixed above), and the top-level weighted mean applies weights exactly once. An earlier version of the aggregation double-counted: domain weights entered both the per-domain score and the overall mean, which would cap an end-to-end assessment well below the coordinator's 0.7 threshold. The aggregation has been fixed to apply the weight only at the top level (equations below); the injected-score tests of Section 5.6 bypass the metric pipeline regardless, because the instruction-level metric extraction remains the stub disclosed above.

Assessment proceeds in three steps:

1. **Domain metric computation.** For each domain, `assess_capabilities` is *specified* to compute a set of `CapabilityMetric` values from the agent's instruction set—e.g., for the reasoning domain, metrics derived from the prevalence and complexity of reasoning-type instructions. As stated above, this extraction is a specified stub in the shipped code (the `_measure_*` functions return 0.0), so exercised assessments derive domain scores from injected metrics.

2. **Domain aggregation.** Metrics are aggregated into a per-domain `DomainScore` as a confidence-weighted mean, matching the shipped `_compute_weighted_score`:

$$s_k = \frac{\sum_{m \in M_k} v_m \cdot \text{conf}_m}{\sum_{m \in M_k} \text{conf}_m}$$

where M_k is the metric set of domain k, v_m the metric value, and conf_m its confidence; an empty metric set scores 0, and a zero total confidence degenerates to the unweighted mean. The domain weight w_k is deliberately absent here—it enters only once, at the top level (an earlier shipped form folded it into s_k as well, double-counting it; Section 5.6's tests were unaffected because they inject `DomainScore` values directly).

3. **Overall score.** `get_overall_score` computes the weighted mean across assessed domains:

$$S(a) = \frac{\sum_k w_k \, s_k}{\sum_k w_k}$$

with $S(a)$ falling back to the unweighted mean when all weights are zero.

Three properties of this design deserve emphasis. First, it is **static rather than live-probing**: assessment is designed to be cheap, deterministic, and side-effect-free, though the shipped metric extraction is the stub disclosed above and exercised scores are injected. Second, it is **compositional**: because scores summarize instructions, fusing or eliminating instructions during compilation (Section 3.4) changes the profile predictably, and a production deployment would re-assess after optimization (the shipped coordinator does not yet do so). Third, it directly serves coordination: the agent-selection predicate in Section 5.6 chooses the highest-scoring agent above a capability threshold for each task, subject to the security policies of the `SecurityMonitor`—with the tests injecting scores directly, as disclosed above.

We state the scope of this subsystem honestly: `CapabilityProfile` assesses *declared instruction-level capability*, which correlates with—but does not measure—end-to-end task performance. Cross-model behavioral comparison at benchmark level is the domain of evaluation suites such as BFCL [17] and AgentBench [29]; the two views are complementary.

4 Implementation

This section presents the implementation and validation of the AgentISA framework, with particular emphasis on the compiler infrastructure and optimization techniques described in Section 3.4.

4.1 Compiler Infrastructure

4.1.1 Instruction Selection Implementation

We *specify* the tree-pattern matching approach of Section 3.4.3 (Definition 3.8) as dynamic programming over the IR tree, in the classical manner of bottom-up instruction selection [33]. **The shipped compiler does not yet implement this selector**: its current selection step is a heuristic matcher—name-overlap scoring over the instruction catalogue—whose compact listing is deferred to the source (`agentflow.core.isa.selection`); the DP listing below is the *specified* optimal selector, and none of the benchmark numbers in Section 5 depend on tree-pattern selection. The specified algorithm is:

```
Input: IR tree T, Pattern set P
Output: Optimal pattern selection S

function MATCH-TREE(T, P)
    costs[n] ← ∞ for all nodes n in T
    Let selected[n] = ∅ for all nodes n in T
```

```
    function MATCH-SUBTREE(node)
        if costs[node] ≠ ∞ then
            return costs[node]
        end if

        min_cost ← ∞
        best_pat ← null

        for each pattern p in P do
            if MATCHES(p, node) then
                curr_cost ← PATTERN-COST(p)
                for each child c of node not covered by p do
                    curr_cost ← curr_cost + MATCH-SUBTREE(c)
                end for

                if curr_cost < min_cost then
                    min_cost ← curr_cost
                    best_pat ← p
                end if
            end if
        end for

        costs[node] ← min_cost
        selected[node] ← best_pat
        return min_cost
    end function

    MATCH-SUBTREE(root(T))
    return (costs, selected)
end function
```

We state the optimality property of the specified algorithm, for patterns whose context constraints are *local* (a pattern's feasibility depends only on the node and its immediate operands):

Theorem 4.1 (Selection Optimality, specified algorithm): Given a set of context-local patterns P and an IR tree T, the MATCH-TREE algorithm computes the minimal covering cost; the covering itself is read off the `selected` table afterwards, by walking from the root and skipping subtrees spanned by the chosen pattern.

Proof: By induction on tree height, assuming a covering of T exists (if no pattern matches a node its cost is ∞ and no covering exists):

1. Base case (a leaf has no children): the recurrence degenerates to a plain argmin over the patterns matching the leaf, which is optimal for a single node.

2. Inductive step: For a node v with children $c_1, \ldots, c_k$:

$$\text{cost}(v) = \min_{p \in P} \left\{ p.\text{cost} + \sum_{\substack{c \in \text{children}(v) \\ c \notin \text{covered}(p)}} \text{cost}(c) \right\}$$

   By the induction hypothesis, each $\text{cost}(c)$ is the minimal covering cost of the corresponding subtree; the recurrence takes each candidate pattern's cost plus the (already optimal) costs of the children the candidate does not span, so the minimum over candidate patterns is the minimal covering cost at v—the minimum over disjoint pattern choices is optimal.

The recurrence sums only over children not covered by the candidate pattern p—summing over all children would double-count the cost of subtrees the pattern spans. Section 3.4.3's per-node argmin is therefore exact for single-node patterns; multi-node patterns require this covering recurrence.

The recurrence requires that a pattern's feasibility at a node not depend on the covers chosen for other subtrees; context constraints that couple sibling subtrees break this optimal-substructure property, which is why the statement restricts to context-local patterns. This theorem is a property of the specified algorithm, not of the shipped heuristic selector; no optimality claim is made for the implementation.

4.1.2 Pipeline Scheduling Implementation

We *specify* the pipeline scheduler following the instruction-scheduling formulation of Section 3.4.3 (Deps and resource capacity) and the correctness conditions of Theorem 3.5 (Section 3.4.2), with hazard detection and resource-constrained issue in the style of classical pipeline scheduling [34]. **This cycle-accurate scheduler is design-level**: the executed runtime path is the sequential and parallel-grouped dispatch produced by the compiler's grouping pass (Section 4.1.3); no benchmark in Section 5 exercises a cycle-accurate schedule. The specified scheduler is:

```
Input: Instruction sequence I
Output: Schedule S with cycle assignments

function SCHEDULE-PIPELINE(I)
    D ← BUILD-DEPENDENCY-GRAPH(I)
    S ← empty schedule
    R ← priority queue of ready instructions

    while D not empty do
        c ← CURRENT-CYCLE()

        // Find newly ready instructions
        N ← GET-READY-NODES(D, c)
        ADD-TO-QUEUE(R, N)

        // Schedule instructions for current cycle
        while R not empty and HAS-RESOURCES(c) do
            i ← POP-HIGHEST-PRIORITY(R)
            slot ← ALLOCATE-RESOURCES(i, c)
```

```
            ADD-TO-SCHEDULE(S, i, slot)
            UPDATE-DEPENDENCIES(D, i)
        end while
    end while

    VERIFY-SCHEDULE(S)
    return S
end function
```

Schedule verification checks the dependency-ordering condition (S2) of the formal model (Theorem 3.5, Appendix C.5), with the edge convention of Section 3.4.3—$(i_1, i_2) \in \text{Deps}$ means i_1 must finish before i_2 starts. The check as written,

$$\forall (i_1, i_2) \in \text{Deps} : \text{slot}(i_2) > \text{slot}(i_1),$$

is equivalent to $start(i_2) \geq end(i_1)$ only under the **unit-slot model** the shipped scheduler assumes: one slot per cycle, every instruction occupying exactly one slot, so $start(i) = \text{slot}(i) \cdot c$ and $end(i) = (\text{slot}(i) + 1) \cdot c$ for cycle length c. If instructions with multi-cycle latencies were admitted, the slot comparison would have to be replaced by an explicit $start/end$ comparison (or slots generalized to intervals); the pipeline scheduler that would need this remains the specified component of Section 4.1.2. S1 (exactly-once dispatch) and S4 (in-order commit) hold by construction of the scheduling loop, and S3 (group non-interference) is enforced by the grouping check.

4.1.3 Static Optimization Implementation

We implement the optimization passes defined in Section 3.4.4. The passes that fire in the benchmarks are fusion (the instruction-combining pass of Section 3.4.4, whose full listing appears below), dead-code elimination (the shipped two-level design of Section 3.4.4), and parallel grouping (described with its measurement in Section 5.3). The shipped rule set registers no peephole patterns, so every peephole obligation below is currently dormant but stated for completeness:

1. **Peephole Optimization**:

```python
class PeepholeOptimizer:
    def optimize(self, instructions: List[Instruction]) -> List[Instruction]:
        patterns = self.load_patterns()
        optimized = []
        i = 0
        while i < len(instructions):
            matched = False
            for pattern in patterns:
                if pattern.matches(instructions[i:i+pattern.length]):
                    optimized.extend(pattern.replace(instructions[i:i+pattern.length]))
                    i += pattern.length
                    matched = True
                    break
            if not matched:
                optimized.append(instructions[i])
```

```
                i += 1
        return optimized
```

Each peephole rule carries a correctness *obligation*, discharged as part of rule definition rather than by the running compiler:

$$\forall p \in \text{Patterns}, w \in \text{Instructions}^{\text{len}(p)} :$$
$$\text{matches}(p, w) \implies \text{sem}(\text{replace}(p, w)) \equiv \text{sem}(w)$$

As stated above, the shipped rule set is empty, so the equivalence checker is dormant; Section 4.3.1 specifies the symbolic-execution checker a populated rule set would require.

2. **Instruction Combining**:

```
def combine_instructions(self, insts: List[Instruction]) -> List[Instruction]:
    combined = []
    current_group = []

    for inst in insts:
        if self.can_combine(current_group, inst):
            current_group.append(inst)
        else:
            if current_group:
                combined.append(self.merge_group(current_group))
            current_group = [inst]

    if current_group:
        combined.append(self.merge_group(current_group))
    return self.verify_combinations(combined)
```

4.2 Runtime System

4.2.1 Dynamic Pipeline Control

Design of the dynamic pipeline controller (**specified, not implemented**—the shipped dynamic machinery is the counter/threshold hot-path detector and the trace re-optimization described in Section 3.4.4 (both discussed in Section 4.2.2; measured in Sections 5.4 and 5.8); a skeleton module exists but is not wired into the runtime):

```
class PipelineController:
    def __init__(self):
        self.hazard_detector = HazardDetector()
        self.resource_monitor = ResourceMonitor()
        self.performance_tracker = PerformanceTracker()

    async def execute_pipeline(self,
                               instruction_stream: AsyncIterator[Instruction]
                              ) -> AsyncIterator[Result]:
        async for inst in instruction_stream:
```

```
        # Hazard detection
        while self.hazard_detector.has_hazard(inst):
            await self.hazard_detector.wait_resolution()

        # Resource allocation
        resources = await self.resource_monitor.allocate(inst)

        try:
            # Execution
            result = await self.execute_instruction(inst, resources)
            # Performance tracking
            self.performance_tracker.record(inst, result)
            yield result
        finally:
            self.resource_monitor.release(resources)
```

Pipeline performance is modeled with the classical pipeline bound—throughput is the reciprocal of the slowest stage's service time:

$$\text{Throughput} = \frac{1}{\max_i \text{service_time}(\text{stage}_i)}$$

4.2.2 Dynamic Optimization Implementation

The *shipped* mechanism is the counter/threshold detector described in Section 3.4.4: a monotone count per exact `path_id`, crossed against a configurable threshold with a last-recorded-success-rate gate. What the benchmarks measure (Sections 5.4 and 5.8) is this mechanism. The following is the *specified* detector—exponential moving average over a window, frequency normalized by recent execution share, and trace formation from hot-path regions; it is **not yet implemented**. Together with the block-granular trace formation below it addresses workloads with trace diversity; the detector itself remains exact-path-keyed (the limitation of Section 3.4.4—that similar but non-identical sequences never accumulate a count—still applies to it):

1. **Hot Path Detection** (specified):

```
Input: Execution path P, Threshold θ, EMA decay γ (γ = 2/(w+1) for window w; γ is unrelated to t
Output: Boolean indicating if P is hot

function DETECT-HOT-PATH(P, θ, γ)
    Let counter be an exponential moving average with decay γ

    function UPDATE-COUNTERS(path)
        h ← COMPUTE-PATH-HASH(path)
        counter[h] ← counter[h] × (1 - γ) + 1

        if IS-PATH-HOT(path) then
            TRIGGER-OPTIMIZATION(path)
        end if
    end function
```

```
    function IS-PATH-HOT(path)
        h ← COMPUTE-PATH-HASH(path)
        freq ← counter[h] / Σ_{h'} counter[h']   // recent execution share, not cumulative total
        stab ← COMPUTE-STABILITY(path)
        return freq > θ and stab > STABILITY-THRESHOLD
    end function

    UPDATE-COUNTERS(P)
    return IS-PATH-HOT(P)
end function
```

The decision rule of the pseudocode is the threshold predicate IS-PATH-HOT (frequency above θ and stability above the stability threshold); it computes no score. A *ranking variant*—specified, not part of the decision rule—can order candidate paths by the heuristic score below, reusing the frequency already normalized in the pseudocode (freq(path) = counter[h]/Σ counter, so no second normalization) blended with recent stability through a static stability weight β:

$$\text{score}(\text{path}) = \text{freq}(\text{path}) \cdot \beta \cdot \text{stab}(\text{path})$$

This is a heuristic score, not a conditional probability.

2. **Trace Formation** (specified):

```
Input: Hot path H
Output: Optimized trace T

function BUILD-TRACE(H)
    T ← empty trace

    function SHOULD-INCLUDE(block)
        return EXECUTION-FREQUENCY(block) > THRESHOLD and
               PREDICTABLE-BRANCHES(block)
    end function

    function SHOULD-SPLIT(block)
        return HAS-SIDE-EXIT(block) or
               EXECUTION-FREQUENCY(block) < MIN-FREQ
    end function

    for each block B in H do
        if SHOULD-INCLUDE(B) then
            APPEND-TO-TRACE(T, B)
        else
            break   // any block not selected terminates the trace
        end if
    end for

    VERIFY-TRACE-PROPERTIES(T)
```

```
    return T
end function
```

Trace validity is *defined* by the conjunction of three properties (H the loop header of the region, tail the last block of the trace); VERIFY-TRACE-PROPERTIES checks them:

$$\text{Valid(trace)} \iff \text{Entry(trace) dominated by } H \wedge \text{Exit(trace) post-dominates tail} \wedge \text{NoSideExit(trace) except tail}$$

4.3 Optimization Verification

4.3.1 Static Analysis

We *specify* verification of the static optimizations from Section 3.4.4 (a symbolic-execution equivalence checker over transformation path pairs). As stated in Section 4.1.3, the shipped peephole table is empty and this obligation is dormant; the soundness theorems (Theorems 3.4–3.6) are conditional on discharging it when rules are introduced (Appendix C.7):

```
Input: Original CFG O, Transformed CFG T
Output: Boolean indicating semantic equivalence

function VERIFY-TRANSFORMATION(O, T)
    // Build symbolic execution paths
    P₁ ← SYMBOLIC-EXECUTE(O)
    P₂ ← SYMBOLIC-EXECUTE(T)

    function VERIFY-EQUIVALENCE(path₁, path₂)
        // σ ranges over the symbolic states Σ explored on both paths
        for each symbolic state σ ∈ Σ do
            if EVALUATE(path₁, σ) ≠ EVALUATE(path₂, σ) then
                return false
            end if
        end for
        return true
    end function

    // Paths must pair one-to-one; a length mismatch means the
    // transformation added or removed a symbolic path — reject.
    if LENGTH(P₁) ≠ LENGTH(P₂) then
        return false
    end if

    // Verify all path pairs, paired by the transformation's path map
    for each (p₁, p₂) in PATH-MAP(P₁, P₂) do
        if not VERIFY-EQUIVALENCE(p₁, p₂) then
            return false
        end if
    end for
```

```
    return true
end function
```

The verification follows the formal model:

$$\text{Equiv}(p_1, p_2) \iff \forall \sigma \in \Sigma : [\![p_1]\!]\sigma = [\![p_2]\!]\sigma$$

4.3.2 Dynamic Analysis

The runtime *monitors* optimizations through a single mechanism (no result-correctness oracle is defined at runtime—correctness of LLM-instruction results is not decidable in general and is delegated to the authoring-time verification of Section 3.3.3; the monitor below is a specified, not shipped, component):

1. **Performance Monitoring**:

```
class PerformanceMonitor:
    def monitor_execution(self,
                          instruction: Instruction,
                          context: Context) -> Metrics:
        start = time.perf_counter_ns()
        result = self.execute(instruction, context)
        end = time.perf_counter_ns()

        return Metrics(
            latency=(end - start),
            resource_usage=self.measure_resources()
        )
```

The evaluation protocol of Section 5.1.3 reports per-metric mean ± standard deviation and makes no inferential (hypothesis-test) claims; no aggregate weighted-mean score is computed.

4.3.3 Empirical Results

The implemented optimizations are validated by the microbenchmark suite of Section 5, which drives the real compiler, cache, and optimization-manager code paths against a mock LLM backend. We keep all numbers in Section 5 and state only the mechanism-to-experiment map here:

Table 1: Mechanism-to-experiment map; every measurement is a controlled-setting result, detailed in Section 5.

Mechanism (implementation status)	Experiment
Result caching (implemented)	Mock-backend caching, 3 skew levels (Section 5.2)
Fusion + DCE static passes (implemented, fire in benchmarks)	Static compilation, 4 levels, 3 lengths (Section 5.3)
Hot-path detection and adoption (implemented)	Dynamic trace re-optimization (Section 5.4)
Pass-level attribution (paired + independent)	Ablation (Section 5.5)
Coordination + security	43 functional tests (Section 5.6)
Cache, compilation, hot-path adoption (live gateway)	Real-gateway validations (Sections 5.7, 5.7.1)
Whole-system composition	Four-task characterization (Section 5.8)

All measurements are controlled-setting results on synthetic workloads (simulated 80 ms per call); production end-to-end claims are out of scope, and Section 5.9 states the threats to validity.

5 Empirical Evaluation

We evaluate AgentISA with a reproducible microbenchmark suite (`examples/benchmarks/isa_microbench.py`) and a functional test suite for the multi-agent layer, under the scope fixed at the end of Section 4.3: Sections 5.2–5.5 run against the mock backend; Sections 5.7–5.8 extend the validation to a live LLM gateway and four system-level tasks; Section 5.9 discusses the threats to validity.

5.1 Methodology

5.1.1 Instrumentation: Mock LLM Backend

All experiments route model inference through a `MockLLMBackend` that (1) counts each `infer` call as one simulated LLM API round trip, and (2) charges a fixed 80 ms simulated latency per call to a simulated clock. The backend never sleeps and never contacts a live model, which makes every "API call" observable, deterministic given a seed, and reproducible without network access or inference spend. Crucially, everything *around* the backend is real: the cache, compiler, and optimizer are the production `agentflow.core.isa` implementations (`InstructionCache`, `InstructionCompiler`, `OptimizationManager`), exercised through the same code paths a deployment would use.

5.1.2 Workloads

Workloads are synthetic and seed-controlled:

- **Caching**: 2,000 tasks drawn from a discrete distribution over 50 instruction templates $\times$ 20 parameter values, with task popularity following a Zipf distribution with exponent $s \in \{0.5, 1.0, 1.5\}$. Increasing s concentrates popularity on fewer tasks, increasing reuse potential.
- **Static compilation**: random instruction sequences of length 50, 200, and 1,000, generated by `synthetic_formal_sequence` (`examples/benchmarks/isa_microbench.py`; the generator is short enough to audit line by line). Each of the first $n-1$ slots is drawn independently: with probability 0.25 a *dead* instruction (side-effect-free, **no declared outputs, hence no consumers anywhere in the sequence**—it is removable by construction, and this removability is planted, not inferred); with probability 0.55 a *pure* instruction (side-effect-free, one declared output v_i, consumed **only by the final sink**—fusable when adjacent to a same-type pure neighbor); otherwise a *side-effectful* instruction (always live, never fused, never removed). The n-th slot is always a side-effectful *final sink* whose dependency list is exactly the set of all pure outputs, so every pure instruction has a downstream consumer and no pure output is deleted as dead. These three attributes are disjoint and jointly exhaustive per slot, and the DCE/fusion targets they plant are what the passes find; the measured reductions are properties of this workload family, not of arbitrary agent programs.
- **Dynamic optimization**: instruction traces of length 100 recorded for up to 200 iterations per seed, with hot-path detection active (threshold 50 recordings). The recording loop terminates at the hot-path trigger—iteration 50 in all seeds—so each seed contributes exactly 51 recordings (the 50 that build the count plus the triggering one).
- **Ablations**: two arms. The primary arm is *paired*: it re-compiles the same sequences as the static-compilation experiment (identical seed base 2,000) with fusion and dead-code-

elimination toggled individually and jointly, yielding a same-input decomposition of the 34.2% headline. The secondary arm is an *independent* draw with the same distribution parameters but seed base 4,000, serving as a replication on fresh sequences (its all-passes total, 35.0%, agrees with the headline within one standard deviation).

5.1.3 Protocol and Statistics

Each synthetic-workload configuration (Sections 5.2–5.5) is run with 5 random seeds; we report mean ± sample standard deviation across seeds. These are **descriptive statistics only**: we make no hypothesis-test claims (no p-values, no significance tests) anywhere in this paper. Five seeds characterize variability across the tested workload instances; they do not establish statistical significance or population-level generality. The naive response-cache baseline of Section 5.2 shares the mock-backend instrumentation and is reported alongside the treatment; a formal *inferential* comparison against full external systems remains out of scope, and no such significance claim is made.

5.1.4 Metric Terminology

The paper reports several distinct quantities that must not be conflated; each is defined once here and used consistently:

- **Effective-instruction reduction**—fewer *effective* instructions in the compiled program (static compilation, Sections 5.3/5.5); a property of the instruction stream, not of execution time.
- **Dispatch-count reduction**—fewer dispatch operations at runtime, including parallel grouping's folding (Section 5.3); a runtime count.
- **Trace shrinkage**—shorter instruction traces after hot-path re-optimization (Section 5.4); the shrink is a trace-length measurement, with the optimized trace's executed-state identity verified separately in the same section.
- **Cache hit rate / call reduction**—the fraction of calls served from the result cache; on this benchmark the call-reduction ratio equals the hit rate by construction (Section 5.2).
- **Simulated latency saving**—avoided-call counts × the mock's fixed 80 ms (Section 5.2); derived, not measured.
- **Measured latency/token saving**—wall-clock time and token counts against the real gateway (Sections 5.7–5.7.1); measured.

For context preservation (Theorem 3.3): *extraction-time context preservation* is the copy invariance of the extraction step; *runtime context validation* is the pre/post-dispatch constraint checking of the monitor. They are different properties with different coverage (Section 3.2.2 discloses the deletion blind spot of the runtime side).

5.2 Experiment 1: Result Caching

Research question. How many simulated LLM API calls does AgentISA's result caching eliminate, as a function of workload skew?

We execute 2,000 tasks per seed, once with caching disabled and once with the `InstructionCache` enabled, and measure simulated API calls at the backend. The reduction ratio and cache hit rate coincide by construction: every avoided call corresponds to exactly one cache hit.

Cache-key contract. The shipped result cache keys an entry on the (instruction, params) pair—deliberately *not* on model identity/version, prompt-template version, tool schema, decoding parameters, or ambient context. This is sound only when those factors are held fixed, which every benchmark here guarantees by construction (single model alias, fixed prompt templates, fixed decoding configuration, fresh cache per arm); the classifier-keyed step of Section 5.7.1 likewise hashes exactly the input dictionary its prompt is built from, under the same fixed-configuration discipline. A deployment that changes any of these factors must invalidate or namespace the cache accordingly (the production cache already isolates by TTL, LRU capacity, and never-cached failures, but it does not detect semantic-configuration drift); modeling cache validity across *varying* model versions or prompts is future work and is not exercised by any number in this paper.

Table 2: Simulated API calls with and without the result cache (2,000 tasks per seed, 5 seeds, mean ± sample standard deviation; reduction equals hit rate by construction).

Zipf exponent s	Calls (no cache)	Calls (cached)	Reduction = hit rate
0.5	2,000	810.4 ± 5.7	59.5% ± 0.3%
1.0	2,000	642.0 ± 3.8	67.9% ± 0.2%
1.5	2,000	408.0 ± 9.4	79.6% ± 0.5%

Naive response-cache baseline. A third arm runs an app-level *response* cache keyed on the exact request payload string—what an exact-match cache in front of the backend (the response-level caching of GPTCache's exact mode, or an HTTP response cache) keys on—implemented as a plain dict with no AgentISA machinery. It issues 810.4 ± 5.7 / 642.0 ± 3.8 / 408.0 ± 9.4 calls at $s = 0.5/1.0/1.5$: **parity with the `InstructionCache` to the digit**, because on this workload the (instruction, params) key is already canonical. Parity is the finding, stated plainly: at the level of call reduction on this workload, the instruction-level cache adds nothing over exact-response caching. What the instruction level adds is the compile-time transformations of Sections 5.3 and 5.5—fusion and dead-code elimination, which reduce the program before any request is issued and which a response cache cannot express because it sees request strings, not instruction semantics. Semantic and embedding-based caching (GPTCache's approximate mode [46]) remains out of scope: no head-to-head against approximate matching is claimed.

Interpretation. The reduction grows monotonically with skew, from 59.5% under mild skew ($s = 0.5$) to 79.6% under heavy concentration ($s = 1.5$). This matches the cache-aware expectation: the upper bound on savings is the probability mass of repeated tasks, and the Zipf exponent directly controls that mass. In simulated latency terms the savings are the avoided-call counts × 80 ms ($s = 0.5/1.0/1.5$: 95,168 ± 455 / 108,640 ± 305 / 127,360 ± 750 ms)—derived, not separately measured.

Two caveats keep this honest. First, these are *simulated* calls against a mock backend: the reduction is exact for this workload family, but real deployments face cache-invalidation, prompt-context, and embedding-variability effects that the mock abstracts away. Second, the absolute number depends on the workload's reuse structure; workloads without repetition (e.g., each task unique) would see zero benefit. What the experiment establishes is the *mechanism and its sensitivity to skew*, not a universal constant.

5.3 Experiment 2: Static Compilation

Research question. How much does the compiler reduce instruction sequences, and what do the different optimization levels contribute?

We compile random instruction sequences at four optimization levels and measure two distinct quantities. We separate them explicitly because they are easy to conflate:

- **Effective instruction count**: the number of instructions that remain in the optimized program after fusion and dead-code elimination. This measures program size.
- **Dispatch count**: the number of independent dispatch operations the runtime issues. Parallel grouping can fold several independent instructions into one dispatch *without changing the effective instruction count*.

Table 3: Effective-instruction-count reduction by optimization level and sequence length (5 seeds, mean ± sample standard deviation; planted dead 0.25 / pure 0.55 workload).

Sequence length	Level	Effective-instruction reduction	Dispatch count
50	NONE	0% ± 0.0	50.0 ± 0.0
50	BASIC	32.8% ± 4.1%	33.6 ± 2.1
50	ADVANCED	32.8% ± 4.1%	19.0 ± 4.1
50	AGGRESSIVE	32.8% ± 4.1%	19.0 ± 4.1
200	NONE	0% ± 0.0	200.0 ± 0.0
200	BASIC	34.2% ± 1.8%	131.6 ± 3.6
200	ADVANCED	34.2% ± 1.8%	74.0 ± 6.3
200	AGGRESSIVE	34.2% ± 1.8%	74.0 ± 6.3
1,000	NONE	0% ± 0.0	1,000.0 ± 0.0
1,000	BASIC	35.4% ± 1.6%	646.0 ± 16.4
1,000	ADVANCED	35.4% ± 1.6%	344.6 ± 31.6
1,000	AGGRESSIVE	35.4% ± 1.6%	344.6 ± 31.6

Interpretation. Three observations:

1. **Effective instruction count reduction is level-stable**: BASIC, ADVANCED, and AGGRESSIVE produce identical reductions (34.2% ± 1.8% at length 200). The fusion and DCE passes implemented at BASIC level already capture the available gains on these synthetic sequences; the higher levels add *scheduling* improvements (dispatch folding), not additional elimination—and on these workloads ADVANCED and AGGRESSIVE coincide, their extra passes finding no further targets; they would differ only on streams carrying combinable patterns. We report this as-is rather than claiming monotonic gains across levels.

2. **Dispatch count drops further with parallel grouping**: at length 200, dispatch falls from 131.6 (BASIC) to 74.0 (ADVANCED) while effective instruction reduction is unchanged. This is the dispatch-folding effect described above: adjacent side-effect-free instructions with disjoint dependencies and outputs grouped into shared dispatches. Readers should interpret the 74.0 figure as a scheduling reduction, not a further program-size reduction. (An earlier implementation folded more aggressively—44.4 at length 200—but could place side-effectful instructions into shared groups whose non-interference was never established; the sounder grouping reported here folds less, and Appendix C.5's dispatch-folding corollary now matches the implementation exactly.)

3. **Gains are stable across sequence lengths**: 32.8%, 34.2%, 35.4% at lengths 50, 200, 1,000 respectively, with the tightest variance at length 1,000 (± 1.6%). Length does not materially change the opportunity structure of these synthetic sequences.

Note again that effective-instruction reduction is a *program-size* metric. It translates to simulated API-call savings only where eliminated or fused instructions carry model inference; our caching experiment measures the complementary reuse effect.

5.4 Experiment 3: Dynamic Hot-Path Optimization

Research question. Does the dynamic optimizer detect and specialize repeatedly executed traces?

We record an instruction trace of length 100 for up to 200 iterations per seed with hot-path detection active; the recording loop terminates at the trigger (iteration 50 in all seeds, 51 recordings per seed: the 50 counted plus the trigger). Once a path is flagged hot, the runtime forms an optimized trace. **This experiment measures the length of that optimized trace**—the benchmark does not execute it. Whether an optimized trace, when actually executed, preserves the task's observable output is established separately, at task level, by the system-level characterization of Section 5.8 (which re-executes hot-path-optimized instruction streams and verifies byte-identical output).

Table 4: Hot-path trace shrinkage per seed (length-100 traces, detection threshold 50 recordings; each seed contributes exactly 51 recordings). The replay column records the executed-state identity check: the optimized trace, re-executed against an independent state, reproduces the original trace's final state exactly in every seed.

Seed	Original length	Optimized length	Shrink ratio	Replay state identical
0	100	30	70.0%	yes
1	100	41	59.0%	yes
2	100	30	70.0%	yes
3	100	36	64.0%	yes
4	100	34	66.0%	yes

Across seeds, hot-path optimization shrinks the trace by **65.8% ± 4.6%**. The seed-to-seed spread (59.0%–70.0%) reflects the variability in which sub-sequences each synthetic trace makes hot; the effect is present in all five seeds. As with the static experiments, this measures trace-length reduction on synthetic traces—workload-controlled evidence that the detection-formation-reoptimization loop functions, not a wall-clock speedup claim.

Replay verification. Each seed's optimized trace *is* re-executed: every instruction reads its declared dependencies from a threaded state dict and returns deterministic bindings for its declared outputs (side-effectful instructions additionally leave per-id markers), and the final state of the optimized trace is compared with that of the original trace executed the same way. Across all five seeds the states are identical (5/5; recorded per seed as `replay_state_identical` in the results file), so the 65.8% shrinkage carries an executed-equality check at trace level—on the deterministic mock, not against a live model. This is the same style of check as Section 5.8's task-level byte-identity, applied to the synthetic traces themselves; what remains untested for these traces is live-model behavior, which no mock experiment can address.

5.5 Ablation: Per-pass Contribution

To attribute the static gains we run two ablations. The primary one is **paired**: it compiles the *same* sequences as Section 5.3 (identical seed base 2000 and generator), with fusion and dead-code-elimination toggled individually and jointly, so per-configuration differences are same-input paired differences—an exact decomposition of the headline rather than a replication across independent draws:

Table 5: Paired per-pass decomposition on the Section 5.3 sequences (length 200, 5 seeds, identical seed base); the fusion + DCE row reproduces the Section 5.3 headline exactly because the inputs are identical.

Configuration	Instruction-count reduction
NONE	0.0% ± 0.0%
fusion only	21.6% ± 1.2%
DCE only	24.5% ± 2.6%
fusion + DCE	34.2% ± 1.8%

The fusion+DCE row is 34.2% ± 1.8%—the headline number itself, confirming that these two passes account for the entire static gain at length 200. The **paired interaction** (fusion+DCE − fusion − DCE, computed per seed) is −11.9% ± 2.0% (per-seed −14.5/−9.5/−13.0/−12.0/−10.5 points): the passes overlap, because dead slots break fusion chains and fused pairs absorb neighbors that DCE would otherwise have removed individually, so the sum of the single-pass gains (46.1 points) overstates the joint effect by 11.9 points.

The secondary ablation is the earlier **independent-draw replication** (seed base 4,000, same distribution): fusion alone 21.9% ± 3.5%, DCE alone 24.1% ± 3.3%, fusion+DCE and all passes 35.0% ± 3.7%. It reproduces the paired structure on fresh sequences—the single-pass shares agree within one standard deviation—so the decomposition is not an artifact of one particular draw. The "all passes" configuration equals fusion+DCE on these workloads, consistent with the level-stability observation in Section 5.3: no additional pass contributes measurable reduction here. We report the absence of additional gain rather than dressing the higher levels in unearned credit. And because the workloads plant dead instructions at rate 0.25 by construction, DCE's per-pass share is near-tautological for this family—it measures that the pass finds the planted dead code, which is precisely its function.

5.6 Multi-Agent Coordination: Functional Validation

The multi-agent layer—`AgentCoordinator`, `SecurityMonitor`, and the collaborative instruction set—is validated functionally by 43 tests, all passing. We present this as **qualitative functional validation**: the tests verify that the coordination behaviors exist and are correct, not that they are fast. No performance number should be inferred from this subsection.

The coordination flow implemented by `AgentCoordinator`:

```
Input: Agent set A, Task list τs
Output: Coordinated solutions S

function COORDINATE-AGENTS(A, τs)
```

```
    for each task T in τs do
        AUTHENTICATE(task issuer)              // SecurityMonitor
        AUTHORIZE(task, role policy)           // SecurityMonitor
        ag ← SELECT-BY-CAPABILITY(A, T)        // CapabilityProfile (Section 3.5)
        plan ← BUILD-EXECUTION-PLAN(T)         // stages + coordination rules
        task-outcome ← success
        for each stage in plan do
            stage-outcome ← EXECUTE-STAGE(stage, ag)
            if stage-outcome failed and fallback(stage) exists then
                stage-outcome ← EXECUTE-FALLBACK(stage)
            end if
            if stage-outcome failed then
                task-outcome ← failed
                break
            end if
        end for
        RECORD-HISTORY(T, task-outcome)
    end for
end function
```

What the 43 tests verify, grouped by subsystem:

1. **Agent registration and role policies** (registration creates policies; permissions differ by role; security requirements differ by role; trust levels are bounded and reward declared specializations).
2. **Task coordination** (successful coordination produces a solution; failed tasks are recorded in history; missing authentication and missing authorization raise; absence of suitable agents raises; agent selection respects a capability threshold; the selected coordinator is the highest-scoring agent).
3. **Execution planning** (plans have stage structure with coordination rules; every stage has fallback keys; fallback strategies are differentiated; a fallback actually executes on stage failure; validators are reasoning-heavy non-coordinator agents).
4. **SecurityMonitor** (valid tasks pass; missing authentication fails; missing authorization fails; violations are recorded with timestamps; declared data sensitivity is enforced; sensitive categories infer a high sensitivity level; benign categories pass at medium; resource access is open by default and restrictable by configuration).
5. **Collaborative instructions** (barrier synchronization waits for the declared number of participants and times out when a peer is missing; width-one barriers resolve immediately; resource-allocation instructions reject over-allocation and serialize concurrent requests; the workflow state machine accepts valid actions, rejects invalid ones and double starts; a multi-agent research-team scenario runs end to end).

These tests exercise the same production modules used elsewhere in the system and are fully deterministic (no LLM calls). They establish correctness of the coordination machinery under the tested conditions. What they do not establish is scalability under contention, crash recovery, or adversarial inputs beyond the policy checks above—all are future work (Section 6).

5.7 Real-LLM End-to-End Validation of Result Caching

Research question. Does the caching mechanism behave identically when the "model" is a real LLM behind a live gateway, and how much *measured* wall-clock time and token usage does it save?

Setup. We replace the mock backend with a real model served by a local Anthropic-protocol gateway (TLS with an internal CA; alias `claude-glm-5.3-flash`, a hosted reasoning-capable GLM model; `max_tokens=64`). Requests are sequential HTTP calls (httpx, no proxy, 90 s timeout), with three bounded retries for transient failures—none occurred. Every other component is the production code path: the same `InstructionCache` protocol, the same instruction objects, the same workload generator and the same instruction pool (50 templates) and parameter space (20 values) as Section 5.2.

Scale. Because each real round trip costs ~2.2 s, we reduce scale from 2,000 tasks × 5 seeds to **200 tasks × 1 seed** per Zipf level (s in {0.5, 1.0, 1.5}); the whole run is 1,040 measured experiment round trips (plus one preflight round trip that validates connectivity before timing begins) in 39.4 minutes. The seed pins the task sequence exactly, so the workload is the same one the expectation below is computed from.

An exactness property. A cache hit depends only on the (instruction, params) pair seen so far—not on what the model returns. The production `InstructionCache` is LRU with a TTL, so in general a hit also depends on wall-clock time and capacity; under this benchmark's configuration none of that can bind: `max_size` = 100,000 with at most 200 distinct keys per arm, `ttl` = 3,600 s with every cache-enabled arm completing in under seven minutes (the no-cache arms run to 7.9 minutes and never consult a cache), a fresh cache per arm, and failed calls never cached. Hit/miss decisions are therefore a pure function of the (instruction, params) sequence, and the hit/miss sequence for a fixed workload is *computable without any backend*. If the mechanism is correct, the real run must reproduce that sequence exactly; at the granularity the artifact records—per-arm hit rates and call counts—it does:

Table 6: Real-gateway cache behavior against the computed expectation (200 sequential requests, single seed, three Zipf levels, single model).

Zipf s	Expected hit rate (computed, no backend)	Observed hit rate	Calls (no cache -> cached)	Call reduction	Wall clock (no cache -> cached, ms)	Wall saved (ms)	Tokens saved
0.5	8.5%	8.5%	200 -> 183	8.5%	472,276 -> 405,754	66,522	1,445
1.0	26.0%	26.0%	200 -> 148	26.0%	455,946 -> 334,597	121,349	4,420
1.5	45.5%	45.5%	200 -> 109	45.5%	450,330 -> 244,316	206,014	7,735

Per-call latency was stable across arms (p50 2.13–2.21 s, p95 3.06–3.54 s). Token savings are gateway-reported `usage` differences: every avoided call saves its input tokens plus the tokens the reasoning model generates before responding (output tokens were the binding constraint of `max_tokens=64` in every call).

Interpretation. Three points. First, the mechanism transfers unchanged: observed hit rates match the computed expectation exactly at every skew level, so the synthetic results of Section 5.2 measure the cache, not an artifact of the mock. Second, avoided calls convert into measured time and tokens: 14.1%, 26.6%, and 45.8% of the no-cache arm's wall clock at s = 0.5, 1.0, 1.5.

These measured savings track the call reduction but are not identical to it, because arms were measured sequentially and per-call latency drifted slightly between arms (the exact quantity is the call reduction; wall and token figures are measurements, not derivations). Third, the *absolute* hit rates here (8.5–45.5%) are lower than Section 5.2's (59.5–79.6%) purely because repetition accumulates with task count: at N = 200 these are the exact computed values for this workload family, and at N = 2,000 the same computation yields the Section 5.2 numbers. The dependence on N is a property of the workload, not the backend.

Scope of the claim. This experiment validates the caching mechanism end-to-end: one gateway, one model, one seed, N = 200, sequential requests, no streaming. The artifact records per-arm aggregate hit rates and call counts; per-position sequence identity follows from the deterministic cache decisions under the conditions above, but is not separately logged. The compiler and dynamic-optimization experiments (Sections 5.3–5.5) are structural—they do not dispatch to the backend at all; Section 5.7.1 below extends the real-gateway validation to them through a cache-mediated protocol, and otherwise their validity rests on the code-path and ablation arguments made there. Nothing in this section is a throughput, availability, or dollar-cost claim.

5.7.1 Real-Gateway Validation of Compilation and Hot-Path Optimization

Research question. Do static compilation and closed-loop hot-path adoption preserve the observable semantics of an instruction stream when the stream's LLM step is served by a real model—and what do the mechanisms measurably save on a live gateway?

Setup. A dedicated harness (`examples/paper_real_llm_compiler_test.py`; tests in `tests/system/test_real_llm_compiler_test.py`) reuses the gateway configuration of Section 5.7 (same base URL, internal-CA TLS, alias `claude-glm-5.3-flash`, sequential httpx, 90 s timeout, three bounded retries) but raises `max_tokens` to 512 (Section 5.7 used 64) so that completions dominate the priced comparison; replies still bind at the cap, as below. The harness runs behind a pluggable backend: the real gateway when reachable, and otherwise a deterministic mock with the same protocol that counts calls, accounts tokens from template sizes, and charges the constant 80 ms simulated call latency—so the identical validation runs offline and online. The task is the service-log triage shape of Section 5.8: a seven-instruction stream with one cacheable LLM-type step (`classify_signature`, one model call per distinct incident-signature batch) and the same kind of planted dead guard (`check_format`), executed over repeated realizations drawn from a seeded truncated-Zipf distribution over a fixed universe of 40 task instances.

Arms and protocol. Arm (a) is the unoptimized authored stream. Arm (b) drives the *same* stream through the production machinery: the closed loop of `OptimizationManager` / `optimize_hot_path` records each realization and adopts the shrunken trace once the path goes hot, and the static `InstructionCompiler` compiles the stream at NONE vs AGGRESSIVE with the dataflow-preservation assertion. Arm (a) runs first, cold, and performs every live round trip; arm (b) then executes the same draws against the result cache that arm (a) warmed. The classify step's cache key is the sha256 of exactly the input dictionary its prompt is built from, so arm (b) issuing *zero* new gateway calls proves it computed identical keys—hence constructed identical prompts and consumed identical values—and its final artifacts must come out byte-for-byte equal; any perturbation of the LLM step's inputs by the optimization would surface as a cache miss (a live call) and/or artifact divergence.

Scale. Tiny scale: 50 task realizations × 1 seed × Zipf s = 1.0. The workload contains 22 distinct signature batches, so the entire run is 23 gateway round trips (one preflight plus 22 live calls, all

issued by arm (a)) in 4.4 minutes of wall time—a bounded validation run, not the 200 × 3-skew sweep of Section 5.7.

Results. All checks pass. Arm (a)—the live reference run—made exactly the 22 first-occurrence calls the backend-free computation predicts and served the other 28 realizations from the result cache (hit rate 56.0%), consuming 4,027 prompt and 11,264 gateway-reported completion tokens in 260.2 s of measured call latency. Arm (b)—compiled, hot-path-adopted—made zero new gateway calls (all 50 realizations hit the warmed keys) and reproduced every artifact byte-for-byte. The hot path was adopted at realization 7 (threshold 5; adoption lands two realizations past the threshold because the counter must strictly exceed it and recording follows execution, as in Section 5.8): realizations 1–6 executed the full seven-instruction stream, realizations 7–50 the shrunken six-instruction form with `check_format` absorbed into its successor by fusion—306 instruction executions against arm (a)'s 350, a 12.6% reduction in executed instructions attributable to the adopted shrunken trace (the per-stream shrink is 14.3%, 7 → 6; the realized saving is diluted by the six full-form realizations before adoption; the static compiler alone reproduces the 7 → 6 shrink at AGGRESSIVE, so the two mechanisms' contributions are not separated) and reported separately from the cache's call savings. Static compilation of the same stream: NONE 7 → AGGRESSIVE 6 effective instructions, dataflow preserved.

What this does and does not show. The byte-identity property is cache-mediated by design: the cached classification values entered the artifacts once, through arm (a)—values that on this alias are fallback-derived, as explained below—and arm (b) is proven to compute the same keys and consume the same values. This validates the stream transformation, not model determinism, which is not guaranteed at temperature 0 and is not claimed. On this gateway alias the reasoning model's replies bind at `max_tokens` in every call—as Section 5.7 reported at `max_tokens=64`, and as a diagnostic call at 1,500 tokens still did—so the classify step's severity values come from its deterministic fallback; what is validated against the live model is the mechanism chain (key computation, cache decisions, call and token accounting, artifact identity), not classification quality. The run covers one gateway, one model, one seed, one skew level, sequential requests, no streaming; multi-model, multi-seed, and real-deployment behavior remain untested.

A priced comparison (costs MODELED—computed from a pricing overlay rather than observed—on measured tokens). The harness prices the measured token counts of the run above under three modes: *no optimization* (every realization re-queries at full price), *AgentISA result caching* (first occurrences only—the measured behavior of both arms), and a *provider prompt cache* (MODELED: every realization queries, but repeat prompts bill their input tokens as prompt-cache reads at 0.1x the input price—configurable—mirroring Anthropic's published prompt-caching read discount, with completions always billed at full price). Two simplifications—omitting Anthropic's 1.25x cache-write premium and its 1024-token minimum cacheable prefix—both favor the provider-cache baseline, so the comparison is conservative for us. List prices are parameterized stand-ins ($3/M input, $15/M output by default) because the serving gateway's actual pricing is unknown. Token counts are measurements, the pricing overlay is a model, and no dollar-savings claim is made:

Table 7: Measured calls and tokens with a MODELED pricing overlay (real-gateway run, s = 1.0, 50 realizations; the price column is a model, not a quote).

Pricing mode (real-gateway run, s = 1.0)	Calls	Input tokens (full / cache-read)	Output tokens	Modeled cost
No optimization	50	9,018 / 0	25,600	$0.4111
AgentISA result caching	22	4,027 / 0	11,264	$0.1810
Provider prompt cache (MODELED)	50	4,027 / 4,991	25,600	$0.3976

On this run's token mix the result cache cuts modeled cost by 56.0%—it removes whole calls, completions included—while the modeled provider prompt cache cuts only 3.3%, because the reasoning model's completion tokens dominate and a discount on input reads cannot touch them; the ranking of the two caching strategies is a function of the input/output token mix, not a constant. On the deterministic mock at paper scale (200 tasks × 1 seed × 3 Zipf levels, template-accounted tokens, same modes and prices) the result cache cuts modeled cost by 80.1%, 81.6%, and 85.1% at s = 0.5, 1.0, 1.5, while the modeled provider prompt cache cuts 28.9–31.4%—there the short completions do not dominate, so the input-side discount matters more; both columns remain MODELED-on-measured in both cases. GPTCache, an external cache layer, is explicitly out of scope this pass; no head-to-head against it is claimed.

What the real-gateway experiments do and do not establish. Three layers, kept separate: (1) *gateway integration correctness*—requests reach the gateway, the cache governs calls, and call/token statistics are recorded faithfully: **established** by Sections 5.7 and 5.7.1. (2) *Workflow semantic preservation*—the optimized stream computes the same cache keys and consumes the same cached values as the unoptimized one, yielding byte-identical artifacts: **established as a cache-mediated identity**, which validates the stream transformation, not model determinism (two independent model invocations could differ; the experiment never compares them). (3) *Real-model task quality*—whether the model's classification output is good: **not established here**; the reasoning alias returned results at the token ceiling and the classify step fell back to a deterministic rule, so no statement about live-model output quality is made anywhere in this paper. The abstract, highlights, and conclusion state only layers (1) and (2).

5.8 Controlled Four-Task System-Level Characterization

The experiments above measure individual mechanisms on synthetic sequences. This section is a *controlled four-task system-level characterization*—a functional demonstration under a fixed closed-loop protocol (same task, 12 realizations, one cacheable step each), not a generalization experiment; the identical 91.7% hit-rate row in the runtime table below is a property of that protocol, not four independent cache observations. To make the mechanisms' composition—and the implemented closed-loop optimization claims of Section 3.4.4—checkable rather than aspirational, we provide a seed-controlled workload characterization (`examples/system_task_workload.py`, `tests/system/test_system_task_workload.py`; the original single-task demonstration lives on as `examples/system_task_demo.py`) built from **four tasks with genuinely different dependency shapes**, each realized end-to-end against sandbox-resident files:

- the original **service-log triage**: a seven-instruction linear stream, described below in full;
- **csv-etl-anomaly**: an eight-instruction fan-out pipeline (ingest CSV shards, schema guard, parse, per-column statistics, per-row signature construction, per-row anomaly triage, aggregate, write);
- **report-merge**: a seven-instruction wide-fan-in task (ingest five markdown reports, header guard, dedupe, per-line signatures, cacheable cross-report consistency check, merge, write);
- **regression-diff**: an eight-instruction two-branch convergence task (parallel baseline/candidate log ingests, guard, normalize, diff, classify hunks, aggregate, write).

The full characterization of all four tasks follows the detailed walk-through of the triage task.

The triage task's instruction stream (seven instructions over sandbox-resident files):

1. `ingest_logs`—read six 400-line log files from the sandbox workspace (sandbox I/O, side-effectful);
2. `check_syntax`—a defensive guard that validates the ingested format; on this workload it never rejects (a *planted dead guard*, standing in for the belt-and-braces checks real pipelines carry);
3. `normalize_entries`—parse and normalize lines;
4. `extract_incidents`—lift normalized entries into incident records;
5. `classify_signature`—triage-classify each incident (LLM-type, cacheable, backed by a deterministic rule-based stand-in that counts simulated model calls);
6. `aggregate_triage`—summarize by severity class;
7. `write_report`—write the triage report to the sandbox (side-effectful).

Configuration. Seed 42; 6 files × 400 lines; 12 repeated realizations of the task; hot-path threshold 5. Every phase is deterministic and re-runnable from the repository (Artifact Availability).

What the triage walk-through shows.

- **Static compilation with structural validation.** NONE compiles to 7 effective instructions / 7 dispatches; AGGRESSIVE compiles to 6 / 6 (14.3% reduction): the dead guard fuses with the following pure step rather than being dropped outright. Crucially, this walk-through *asserts the compiled stream's dataflow validity*—every step whose output is consumed downstream must survive compilation, and the only eliminable step is the guard—so the reported shrinkage cannot silently come from dropping live producers.
- **Reuse across repeated realizations.** The first realization makes 50 (simulated) model calls; all eleven subsequent realizations make zero (result cache hit rate 91.7% on the classification step). All 12 reports are byte-identical.
- **Closed-loop hot-path adoption.** The repeated stream crosses the hot-path threshold at the sixth recording (the counter must strictly exceed the threshold, and recording follows execution, so adoption lands at realization threshold+2); the runtime then *adopts* the optimized form automatically, and repeats 7–12 execute the shrunken 6-instruction stream (instruction executions per realization: `[7,7,7,7,7,7,6,6,6,6,6,6]`). Executed on a fresh workspace, the optimized form reproduces the report byte-for-byte—the optimization is semantics-preserving on the real task, not only on synthetic sequences.
- **Full-stack determinism.** A completely fresh stack—new instructions, new cache, new workspace, same seed—reproduces the report byte-for-byte (SHA-256 equality).

Calibrating the synthetic headline. The dynamic hot-path experiment of Section 5.4 reports 65.8% trace shrinkage on synthetic traces with planted dead instructions; the four realistic tasks

show **12.5–25.0%** (one planted guard in a seven- or eight-instruction stream, plus one additional live-pure fusion pair in csv-etl-anomaly). We state both figures rather than privilege either: the 12.5–25.0% number reflects the dead-check density we planted (one guard per seven- to eight-instruction stream), so it calibrates the shrink to a stated density assumption, not to observed ordinary-pipeline waste; the synthetic figure is an upper bound under the skew/planting parameters we control, not a typical-case estimate. Developing this demonstration also served its audit purpose: constructing a realistic (non-synthetic) dependency shape exposed two grouping/fusion soundness gaps in the compiler—side-effectful instructions opening dispatch groups, and fused instructions losing their external dependencies—which are now fixed with regression tests; the dispatch-folding numbers of Section 5.3 reflect the corrected (more conservative) grouping.

The four-task characterization. All four tasks run the identical closed-loop battery (`examples/system_task_workload.py`, `characterize()`, 12 repeated realizations each, hot-path threshold 5, one planted dead guard each, one cacheable LLM-type step each backed by a deterministic rule stand-in that counts simulated model calls):

Structure (compile-time):

Table 8: Compile-time structure of the four tasks (12 realizations each; shrink is the NONE → AGGRESSIVE effective-instruction reduction).

Task (shape)	Instrs	Effective count NONE → AGGRESSIVE	Shrink	Dead guard
log triage (linear, 7)	7	7 → 6	-14.3%	`check_syntax`
csv-etl-anomaly (fan-out, 8)	8	8 → 6	-25.0%	`check_schema`
report-merge (wide fan-in, 7)	7	7 → 6	-14.3%	`check_headers`
regression-diff (two-branch, 8)	8	8 → 7	-12.5%	`check_formats`

Execution (runtime, 12 realizations each):

Table 9: Runtime behavior of the four tasks (12 realizations each; classification-type steps are backed by deterministic rule stand-ins).

Task	Calls 1st / re-runs	Cache hit	Hot-path adopted	Reports byte-identical
log triage	50 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)
csv-etl-anomaly	14 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)
report-merge	16 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)
regression-diff	240 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)

Every task passes the same structural assertions. Static compilation preserves the dataflow: the guard is never dropped outright (the eliminated set is empty in all four tasks), and the reduction

is fusion—the guard merges into its successor. In csv-etl-anomaly a second adjacent pure pair (per-column statistics with row-signature construction) also fuses, which is why its shrink reaches 25.0% with a single planted guard. The first realization pays all model calls and every subsequent realization hits the result cache; the 91.7% hit rate is 11 of 12 realizations by construction (one cacheable step, identical seeded draws), so the four rows share it structurally, not as independent observations. The hot path is adopted at realization 7 in all four tasks (full form for realizations 1–6, shrunken form for 7–12), and the optimized form reproduces the report byte-for-byte on a fresh workspace, as does a completely fresh stack—new instructions, new cache, new workspace, same seed (SHA-256 equality).

The mechanisms behave uniformly across the four dependency shapes: reuse and closed-loop adoption are invariant under the fixed protocol (12 seeded realizations, threshold 5, one cacheable step per task), while the guard/fusion shrink varies with stream structure (12.5–25.0%)—the compile shrink is the only shape-dependent quantity, which is what one wants from an infrastructure claim. These remain four seed-controlled, hand-constructed tasks, not a production workload study; what the characterization adds over the original single-task demonstration is evidence that the composition behavior is not an artifact of one stream's structure.

5.9 Limitations and Threats to Validity

We enumerate the limitations of the evaluation explicitly:

1. **Mock backend, not a live LLM.** *The mock's limits:* it charges a constant 80 ms per simulated call and returns deterministic responses. *What the mock itself covers beyond counting:* the hot-path replay verification of Section 5.4 re-executes each optimized trace and checks final-state identity (5/5 seeds), and the naive-cache baseline of Section 5.2 runs on the same instrumentation. *What the real-gateway experiments cover:* against a live gateway, the cache reproduces the computed hit/miss statistics and converts avoided calls into measured time and tokens (Section 5.7), and the compiled, hot-path-adopted stream issues zero new gateway calls with byte-identical artifacts (Section 5.7.1)—a cache-mediated identity validating the stream transformation, not model determinism. *Still not covered:* multi-model and multi-seed behavior, real deployment, caching under model nondeterminism *across prompt variations*, streaming responses, provider rate limits, and live-model task quality (the four-task characterization's classification steps are deterministic rule-based stand-ins, and the 5.7.1 gateway run fell back to the deterministic classifier).

2. **Synthetic workloads.** Zipf-distributed synthetic tasks are a reasonable model of skewed agent workloads, but they are a model, and the task-level characterization of Section 5.8 only partially addresses this. The cache-reduction figures (59.5–79.6%) are exact for these workload families and should be read as *skew-sensitivity curves*, not as universal savings. A deployment with uniformly novel tasks would see near-zero cache benefit. Section 5.8 adds a four-task, four-shape system-level characterization (linear, fan-out, wide-fan-in, and two-branch convergence streams); its 12.5–25.0% per-task compile shrink calibrates the 65.8% synthetic headline downward, and its uniform reuse and closed-loop adoption under the fixed protocol (12 seeded realizations, threshold 5) show the composition behavior is not an artifact of one stream's structure—but the four tasks remain seed-controlled, hand-constructed characterization streams, not a production workload study.

3. **No production end-to-end evaluation.** We have not deployed AgentISA in a production setting. Consequently this paper contains no latency-under-load, throughput, availability, or

dollar-cost measurements, and we make no such claims.

4. **No real cost data (token usage measured; pricing now MODELED on measured tokens).** All latency figures in Sections 5.2–5.5 are simulated (avoided calls × 80 ms). Section 5.7 measures real wall-clock time and gateway-reported token usage at reduced scale, and Section 5.7.1 prices its own measured tokens (its 22-call reference run) under three modes—no optimization, AgentISA result caching, and a modeled provider prompt cache with a parameterized cache-read discount (default 0.1, after Anthropic's published prompt-caching economics). The resulting costs are MODELED-on-measured: token counts are measurements, prices are configurable list-price stand-ins (the serving gateway's actual pricing is unknown), and no dollar-savings claim is made. Section 5.2's baseline is an *exact-match* response cache, and its parity finding is reported; a head-to-head against *approximate* (semantic/embedding) caching (GPTCache's approximate mode—explicitly out of scope this pass) and a priced deployment study under real provider pricing remain future work.

5. **Program-size metrics, not wall-clock time.** Effective-instruction reduction and dispatch-count reduction are structural metrics. They correlate with—but do not equal—wall-clock improvements, which depend on per-instruction cost, parallelism available in the runtime, and dispatch overhead.

6. **Descriptive statistics only.** Five seeds per configuration in the synthetic experiments (Sections 5.2–5.5); the real-gateway and system-level runs are single-seed by design. No hypothesis tests, confidence intervals, or significance claims are made. The standard deviations indicate stability across seeds, nothing more.

7. **Functional, not performance, validation of coordination.** The 43 multi-agent tests verify behavioral correctness under deterministic conditions, not throughput or scalability.

Despite these limitations, the evaluation achieves its purpose: it demonstrates, under fully disclosed and reproducible conditions, that each optimization mechanism is implemented, active, and produces the claimed structural effects on workloads designed to expose them.

6 Discussion

6.1 System Limitations and Challenges

1. **Instruction-set coverage**
   - Certain complex reasoning patterns are not yet naturally expressible; domain-specific instruction extensions are likely needed
   - Maintaining consistency of pre/post-conditions and typing rules across extensions remains an open engineering challenge
2. **Evaluation Scope**
   - As detailed in Section 5.9, our quantitative evidence is primarily microbenchmark-based (Sections 5.2–5.5), supplemented by real-gateway spot checks (Sections 5.7 and 5.7.1) and a seed-controlled four-task characterization (Section 5.8)
   - Production end-to-end evaluation—including real provider pricing, rate limits, and failure modes—is required before any deployment claim
3. **Performance Bottlenecks**
   - Cache memory consumption under large instruction universes
   - Synchronization overhead in barrier-coordinated multi-agent execution

- Recompilation cost when hot-path thresholds are misconfigured
- The shipped hot-path detector keys on exact sequence identity, so trace diversity (similar-but-not-identical sequences) never accumulates counts; the EMA-based windowed detector specified in Section 4.2.2 is the designed remedy
- Hot-path state (cached sequences, counters, statistics) is currently unbounded; production use needs an eviction policy and a cap on tracked paths

4. **Integration Challenges**
   - Adapter maintenance as provider function-calling surfaces evolve
   - Version management of learned instructions across LLM updates

6.2 Security and Privacy Considerations

AgentISA's implemented security layer is the `SecurityMonitor` of Section 5.6, which enforces task-level authentication, role-based authorization, data-sensitivity classification, and configurable resource-access policies; the coordinator consults it before every assignment, and violations are recorded with timestamps. One shipped default deserves an explicit call-out: **resource access is open by default** and restrictable by configuration—a fail-open stance that shifts the security burden onto whoever writes the policy set. A hardened deployment should invert it to fail-closed (deny on unregistered resources, unknown sensitivity, or incomplete policy), which the current checks do not do; the 43 functional tests validate the implemented checks under configured policies, not the default posture, and nothing in this paper constitutes a security certification. Beyond what is implemented, we identify the following as design considerations for hardening the architecture—not as claims about the current system:

1. **Data privacy**: instruction-level access control exists via authorization policies; end-to-end data-flow tracking and privacy-preserving computation remain future work (Section 6.3.6).
2. **System security**: instruction pre/post-condition verification (Section 3.3.3) checks pure instructions' contracts against bounded state spaces, reducing mis-execution risk there; secure multi-agent communication channels and audit-log integrity are not yet addressed.
3. **Compliance**: deployments in regulated domains will require externally audited controls; the current system provides enforcement hooks, not certification.

6.3 Future Research Directions

Readers pressed for time need only the first tier. **Directly motivated by this paper's exposed gaps:** 6.3.1 (unifying the formal semantics and discharging the conditional theorems—the response to Sections 3.3.3 and C.7's specified-vs-shipped split), 6.3.4 (calibrating capability scores against end-to-end performance—the response to Section 3.5's stub disclosure), and the real-workload evaluation named in 6.3.3's first item (the response to Section 5.9). **Longer-term, with no current implementation link:** 6.3.2 (self-modifying and context-aware instruction sets—their formulas are sketches, as marked inline), 6.3.5 (emergence metrics, cross-agent transfer), and the privacy composition of 6.3.6. We keep the longer-term items brief and flag them as such rather than inflating them into proposals.

6.3.1 Theoretical Foundations

1. **Formal Semantics of Agent Instructions**

   Consolidating Section 3.1 and Appendix B into a single standalone semantics document—one formal framework covering the instruction language, state space, reduction rules, and type

system—is the foundational next step; nothing in it is new machinery, and the composition here is what a journal-length treatment would spell out.

2. **Compositional Reasoning**

   Development of compositional proof systems for agent behaviors:

$$\frac{\{P_1\}S_1\{Q_1\} \quad \{P_2\}S_2\{Q_2\} \quad \text{disjoint}(S_1, S_2)}{\{P_1 \wedge P_2\}S_1 \parallel S_2\{Q_1 \wedge Q_2\}}$$

   The disjointness side condition mirrors the $\parallel$-typing rule of Definition 3.6; without it the conjunction of preconditions does not guarantee either component's precondition during interleaving. This enables formal verification of complex agent systems through:

   - Modular verification
   - Parallel composition rules
   - Temporal property preservation

3. **Type Theory for Agent Capabilities**

   A capability-based type system:

```
Type Capability = {
  Basic: {Perception, Action, Memory}
  Composite: Basic → Basic
  Emergent: P(Composite)
}
```

   With typing rules:

$$\frac{\Gamma \vdash a : \text{cap}_1 \quad \Gamma \vdash b : \text{cap}_2}{\Gamma \vdash a \circ b : \text{cap}_1 \otimes \text{cap}_2}$$

6.3.2 Advanced Instruction Set Evolution

1. **Self-Modifying Instructions**

   Formal model for instruction evolution (f is left abstract—the sketch names the inputs, not a concrete update rule):

$$I_{t+1} = f(I_t, \text{Performance}_t, \text{Context}_t)$$

   With constraints:

   - Semantic preservation
   - Performance monotonicity
   - Safety guarantees

2. **Context-Aware Optimization**

   Programmatic sketch (primitives like the performance model and variant generator are future work, not implemented machinery):

Context-aware optimization—for each observed context, predict the current instruction set's performance against a threshold, and if it falls short, select a best rewrite variant for that context and accumulate it into the set, returning the accumulated set:

```
function OPTIMIZE-CONTEXT(I, C)
    Let M be a context-performance model, R the rewrite rules
    for each context c in C do
        if PREDICT-PERFORMANCE(M, I, c) < THRESHOLD then
            candidates ← GENERATE-VARIANTS(I, R)
            I ← SELECT-BEST(candidates, c)
            UPDATE-MODEL(M, I, c)
        end if
    end for
    return I
end function
```

3. **Cross-Model Instruction Mapping**

   Universal mapping framework:

   $$\phi_{A \to B} : I_A \to I_B$$

   Target properties (stated as goals, not delivered guarantees):

   - Semantic preservation
   - Performance bounds
   - Capability awareness

6.3.3 Theoretical Performance Bounds

1. **Asymptotic Analysis**

   A natural next step is deriving cost bounds per composition operator—e.g., when does iterative composition i_1^* admit a closed-form cost model, and under what reuse structure does caching yield amortized $O(1)$ effective cost per task? Our microbenchmarks (Section 5) measure the constants empirically; the corresponding asymptotic theory remains open.

2. **Resource Consumption Models**

   A calibrated resource model:

   $$R(i) = \alpha C_{cpu}(i) + \beta C_{mem}(i) + \gamma C_{io}(i)$$

   where α, β, γ are platform-specific coefficients that should be fitted against real deployments rather than assumed.

6.3.4 Capability Assessment: Extensions

The instruction-level capability assessment subsystem (`CapabilityProfile`, Section 3.5) ships in part—aggregation and the coordinator's threshold selection are implemented and used, while instruction-level metric extraction remains a specified stub. Two extensions remain future work:

1. **Benchmark-calibrated metrics**: correlating static, instruction-derived domain scores with end-to-end benchmark performance, in the manner of function-calling evaluations [17] and agent benchmarks [29], so that weights w_k are empirically grounded rather than configured by hand.
2. **Cross-model comparison matrices**: extending the per-agent profile to a pairwise comparison framework $\Delta(M_1, M_2)$ over capability dimensions, supporting systematic model selection for a given instruction workload.

6.3.5 Emergent Behavior Analysis

1. **Collective Intelligence Modeling**

   The open question is a formal metric for when multi-agent composition exceeds the sum of its members' capabilities—what emergence would have to mean measurably, rather than an appeal to an undefined φ over unioned capability sets.

2. **Learning Transfer Mechanisms**

   Knowledge transfer across agents composes the mining-extraction-validation pipeline of Section 3.3 rather than introducing new machinery: patterns mined from a source agent's traces would be adapted to a target agent's instruction set and re-validated (threshold and success-rate checks) before promotion. What Section 3.3 does within one agent's traces, this direction would do across agents; the validation obligation transfers unchanged.

6.3.6 Security and Privacy Extensions

1. **Information Flow Control**

   Formal security model:

   $$\text{Secure}(P) \iff \forall s_1, s_2 : L(s_1) = L(s_2) \implies L([\![P]\!]s_1) = L([\![P]\!]s_2)$$

   where L assigns security levels to states and outputs—the classical noninterference property, stated here for terminating runs (divergence and channel leakage out of scope).

2. **Privacy-Preserving Computation**

   A direction, not a delivered mechanism: if agent computations over sensitive data were routed through per-step differentially private mechanisms, the overall loss would follow a *composition theorem* over the per-mechanism (ϵ_i, δ_i) parameters. We deliberately state no bound here: which composition theorem applies, and which concrete mechanisms (hence parameter values) the runtime would expose, are open design choices—any formula written now would be decoration rather than a guarantee. The direction is listed because the instruction contract is the natural place to carry such annotations once the mechanisms exist.

These directions form a roadmap for moving AgentISA from controlled-setting-validated infrastructure to a deployable, formally verified agent platform. None of the frameworks above is implemented today; Sections 6.3.3 and 6.3.4 are stated precisely enough to be falsifiable, while the remaining sketches are programmatic and guide subsequent work.

7 Conclusion

This paper presented AgentISA, an instruction set architecture that treats LLM agent behavior as a first-class compilable artifact. Our contributions are:

1. A formal instruction set with compositional semantics, a type system, and pre/post-conditions, with sanity results stated as design rationale (Turing completeness as negative certification, Theorem 3.1; pure-core type safety, Theorem 3.2; extraction-time context preservation, Theorem 3.3) and semantic-preservation arguments for the optimization and scheduling transformations (Theorems 3.4–3.6).
2. Agentic Calling, which lifts provider-specific function calling into reusable, provider-independent, cacheable agent instructions.
3. Trace-based instruction learning based on frequency mining and deterministic action-signature grouping of execution traces, with developer-controlled promotion and composite replay.
4. A compiler and runtime applying classical optimizations—fusion, dead-code elimination, scheduling, and dynamic hot-path specialization—to agent instruction streams.
5. A multi-agent coordination layer with capability-based selection and a security monitor enforcing authentication, authorization, and data-sensitivity policies.

The evaluation shows the optimizations are real and measurable under controlled conditions: 59.5–79.6% simulated-call reduction from caching, 34.2% instruction-count reduction from static compilation, 65.8% trace shrinkage from hot-path optimization (Sections 5.2–5.5), real-gateway validations of the cache and the compiled pipeline (Sections 5.7 and 5.7.1, gateway-integration and cache-mediated-identity layers only), a four-task system-level characterization (Section 5.8), and 43 functional tests for the multi-agent layer. These are microbenchmark and controlled-setting results, not production end-to-end measurements; the threats to validity of Section 5.9 stand.

Implementation status of the contributions. For auditability, the five contributions above split by status: *implemented and exercised by the benchmarks*—the instruction set and contracts, Agentic Calling, result caching, fusion/DCE/grouping, hot-path detection and adoption, the coordination layer with injected-score capability selection, and the security monitor (43 tests); *implemented with known limits*—the bounded checker (pure instructions, bounded state spaces only), runtime context validation (deletion blind spot disclosed), and pattern mining with composite replay (developer-controlled promotion, no generalization beyond recorded parameters); *specified, not implemented*—tree-pattern instruction selection, cycle-accurate pipeline scheduling, symbolic-equivalence checking, and capability metric extraction. Theorems 3.4–3.6 are conditional arguments for the implemented passes; the specified components carry no verification claim.

AgentISA's longer-term significance is architectural: by giving agent behavior an instruction-level contract, it makes agent systems amenable to the same discipline that transformed hardware and compiler engineering—precise specification, portability across backends, and semantics-preserving optimization. Establishing the corresponding gains on production workloads, and hardening the conditional sanity results and semantic-preservation arguments into mechanically verified implementations, is the immediate next step.

Artifact Availability

All artifacts are available in the repository accompanying this submission (anonymized for review). Reproducibility anchors:

- **Version**: every number in Section 5 is generated from the exact repository state whose build produces this PDF; the artifact's `git rev-parse HEAD` at submission time is recorded in the repository's top-level `REPRODUCING.md`, alongside the dependency lock file and the Python/system versions used for the reported runs.
- **Clean-environment regeneration**: from a fresh clone, `python -m pytest tests/core/isa/test_coordination.py tests/core/instructions/test_agent_coordination.py tests/system/ -q` reproduces the functional-test counts, and the commands listed at the end of this section regenerate each results table from scratch (including the real-gateway runs, whose configuration—endpoint aliasing, model id, decoding parameters—is documented inline in the harness with a mock-backend fallback for environments without gateway credentials).
- **Claim-to-code map**: `REPRODUCING.md` also carries a table mapping each paper claim to the implementing module, the test that exercises it, and the raw-results file that backs its numbers, flagging the specified-only components (symbolic-equivalence checker, tree-pattern selection, cycle-accurate scheduling, capability metric extraction) explicitly.

The artifact contents are:

- the AgentISA implementation (`agentflow.core.isa`);
- the microbenchmark suite (`examples/benchmarks/isa_microbench.py`);
- the real-LLM benchmark (`examples/benchmarks/isa_real_llm_bench.py`);
- the system-level task demonstration (`examples/system_task_demo.py`) and its raw results (`examples/system_task_results.json`);
- the four-task workload characterization harness (`examples/system_task_workload.py`) and its tests (`tests/system/test_system_task_workload.py`);
- the real-gateway compiler/hot-path harness (`examples/paper_real_llm_compiler_test.py`) with its offline-mock and real-gateway results (`examples/paper_real_llm_compiler_results_mock.json`, `examples/paper_real_llm_compiler_results_real.json`) and tests (`tests/system/test_real_llm_compiler_test.py`);
- the raw results (`examples/benchmarks/isa_microbench_results.json` and `examples/benchmarks/isa_real_llm_results.json`);
- the 43 multi-agent functional tests (`tests/core/isa/test_coordination.py` and `tests/core/instructions/test_agent_coordination.py`).

The Section 5.2–5.5 numbers regenerate with:

```
./venv/bin/python examples/benchmarks/isa_microbench.py \
    --experiments cache compiler dynamic ablation --seeds 5
```

The Section 5.7 real-gateway run regenerates with:

```
./venv/bin/python examples/benchmarks/isa_real_llm_bench.py \
    --tasks 200 --seeds 1 --zipf 0.5 1.0 1.5 --max-tokens 64 \
    --output examples/benchmarks/isa_real_llm_results.json
```

(requires a reachable gateway; the base URL, API key, and CA certificate are passed per deployment). The Section 5.7.1 harness regenerates with:

```
./venv/bin/python examples/paper_real_llm_compiler_test.py \
    --backend real --scale paper
```

(the `mock` backend reproduces the offline variant; `--scale paper` selects the 50-realization, 40-instance protocol of the paper run). The Section 5.8 demonstration (and its saved results) regenerates with:

```
./venv/bin/python examples/system_task_demo.py \
    --seed 42 --files 6 --lines-per-file 400 --repeats 12 \
    --hot-threshold 5 --output examples/system_task_results.json
```

and its test suite runs with `pytest tests/system/test_system_task_demo.py`.

The Section 5.7 numbers regenerate against any Anthropic-protocol gateway (configure `ANTHROPIC_BASE_URL` and `ANTHROPIC_AUTH_TOKEN`; the paper's run used a local alias for a hosted GLM reasoning model) with:

```
./venv/bin/python examples/benchmarks/isa_real_llm_bench.py \
    --tasks 200 --seeds 1 --zipf 0.5 1.0 1.5 --max-tokens 64
```

References

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, Dario Amodei. 2020. Language Models are Few-Shot Learners. NeurIPS 33. arXiv:2005.14165.

[2] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, Noah Fiedel. 2023. PaLM: Scaling Language Modeling with Pathways. JMLR 24(240):1–113. arXiv:2204.02311.

[3] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774.

[4] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971.

[5] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, Quoc V. Le. 2022. Finetuned Language Models Are Zero-Shot Learners. ICLR 2022. arXiv:2109.01652.

[6] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, Ryan Lowe. 2022. Training language models to follow instructions with human feedback. NeurIPS 35. arXiv:2203.02155.

[7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 35, pp. 24824–24837. arXiv:2201.11903.

[8] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. NeurIPS 35. arXiv:2205.11916.

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629.

[10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 36. arXiv:2303.11366.

[11] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 36. arXiv:2305.10601.

[12] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.

[13] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 36. arXiv:2302.04761.

[14] Shishir G. Patil, Tianjun Zhang, Xin Wang, Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.

[15] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. ICLR 2024. arXiv:2307.16789.

[16] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. NeurIPS 36. arXiv:2303.17580.

[17] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. ICML 2025. arXiv:2408.12946.

[18] Anthropic. 2024. Introducing the Model Context Protocol. Anthropic announcement, November 25, 2024.

[19] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. (Final version COLM 2024.)

[20] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. ICLR 2024. arXiv:2308.00352.

[21] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, Jie Zhou. 2024. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. ICLR 2024 (Poster). arXiv:2308.10848.

[22] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. ACL 2024, pp. 15174–15186. arXiv:2307.07924.

[23] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. NeurIPS 36. arXiv:2303.17760.

[24] Significant Gravitas. 2023. AutoGPT. GitHub repository. https://github.com/Significant-Gravitas/AutoGPT

[25] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. arXiv:2310.03714.

[26] Harrison Chase. 2022. LangChain. GitHub repository. https://github.com/langchain-ai/langchain

[27] Jerry Liu. 2022. LlamaIndex. GitHub repository. https://github.com/run-llama/llama_index

[28] Lingjiao Chen, Matei Zaharia, James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176.

[29] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. ICLR 2024. arXiv:2308.03688.

[30] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, Thomas Scialom. 2023. GAIA: a benchmark for General AI Assistants. arXiv:2311.12983. (Final version ICLR 2024.)

[31] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, Subbarao Kambhampati. 2023. PlanBench: An Extensible Benchmark for Evaluating Large Language Models

on Planning and Reasoning about Change. NeurIPS 2023 Datasets and Benchmarks Track. arXiv:2206.10498.

[32] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues? ICLR 2024. arXiv:2310.06770.

[33] Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. 2006. Compilers: Principles, Techniques, and Tools (2nd ed.). Addison-Wesley.

[34] John L. Hennessy, David A. Patterson. 2017. Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.

[35] David A. Patterson, Andrew Waterman. 2017. The RISC-V Reader: An Open Architecture Atlas. Strawberry Canyon.

[36] Benjamin C. Pierce. 2002. Types and Programming Languages. MIT Press.

[37] Xavier Leroy. 2009. Formal verification of a realistic compiler. Communications of the ACM 52(7):107–115.

[38] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180.

[39] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI '22, pp. 521–538.

[40] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 35. arXiv:2205.14135.

[41] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation—A KVCache-centric Architecture for Serving LLM Chatbot. FAST '25, pp. 155–170.

[42] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devesh Tiwari, Vijay Gadepally. 2023. From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference. arXiv:2310.03003.

[43] Hanne Riis Nielson, Flemming Nielson. 1992. Semantics with Applications: A Formal Introduction. Wiley. (Springer 2007 retitled edition: Semantics with Applications: An Appetizer.)

[44] Michael Sipser. 2012. Introduction to the Theory of Computation, 3rd edition. Cengage Learning. Chapter 3 (the Church–Turing thesis and equivalence of computational models).

[45] Vasanth Bala, Evelyn Duesterwald, Sanjeev Banerjia. 2000. Dynamo: A Transparent Dynamic Optimization System. PLDI 2000, pp. 1–12. <https://doi.org/10.1145/349299.349303>

[46] Zilliz and community contributors. 2023. GPTCache: An Open-Source Library for Specifying and Implementing Semantics-Aware Caches for LLM Applications. Open-source release and technical report. https://github.com/zilliztech/GPTCache.

[47] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, Ying Sheng.

2024. SGLang: Efficient Execution of Structured Language Model Programs. NeurIPS 37. arXiv:2312.07104.

A Proof of Theorem 3.1 (Turing Completeness)

A.1 Strategy

We prove that AgentISA can simulate any program in the WHILE language over the natural numbers. WHILE programs compute exactly the partial recursive functions—WHILE is Turing equivalent [43, 44]—so a sound simulation of WHILE by AgentISA establishes Turing completeness. The argument is a sufficiency (simulation) argument over instruction classes: the instruction classes of Definition 3.1 must jointly provide unbounded storage, sequential composition, conditional branching, and unbounded iteration. We show the control-flow class $\mathcal{C}$ and the state-management class $\mathcal{S}$ suffice; neither effectful class (the LLM-interaction class $\mathcal{L}$ nor the resource class $\mathcal{R}$) is needed, making the result conservative with respect to the full instruction set.

A.2 Source Language

Let WHILE be the language

$$P ::= x := x{+}1 \mid x := x{-}1 \mid x := y \mid \texttt{skip} \mid \texttt{if } x = 0 \texttt{ then } P_1 \texttt{ else } P_2 \mid \texttt{while } x \neq 0 \texttt{ do } P \mid P_1; P_2$$

over variables ranging over $\mathbb{N}$ (with predecessor cut off at 0), with the standard small-step semantics. Configurations are pairs $\langle P, \sigma \rangle$ with store $\sigma : Var \to \mathbb{N}$.

A.3 Encoding

(i) Store. A WHILE store σ is represented by the AgentISA state $s_\sigma \in \Sigma$, in the component managed by the state-management instructions. Each variable x occupies a dedicated cell; cells hold unbounded naturals, so Σ provides an unbounded store—the crucial ingredient of the sufficiency argument.

(ii) Instruction semantics (as used here). The state-management component of Σ carries, alongside the variable cells, an **evaluation stack**; the five encoding instructions below operate on that stack. Five instructions suffice for the encoding, with the following operational meaning:

- `load` x: pushes the contents of the cell for x onto the evaluation stack;
- `store` x: pops the top of the evaluation stack into the cell for x;
- `update` (f): pops the top of the stack, applies the unary arithmetic function f, and pushes the result; `update`(-1) denotes the *truncated predecessor* $\lambda n.\ \max(n-1, 0)$, matching WHILE's cutoff semantics on $\mathbb{N}$;
- `branch` (b, Q_1, Q_2): pops the top of the stack; if predicate b holds, execute Q_1, else execute Q_2;
- `loop` (Q): execute Q; if its final `branch` returns to the loop head, repeat, else exit—that is, `loop` repeats its body for as long as the body's trailing test succeeds.

(iii) Statements. Each WHILE statement compiles to the AgentISA sequence below (notation defined after the table):

Table 10: Compilation of WHILE statements to AgentISA sequences ($\hat{P}$ is the compilation of P).

WHILE construct	AgentISA compilation
$x := y$	`load` y; `store` x
$x := x + 1$	`load` x; `update`$(+1)$; `store` x
$x := x - 1$	`load` x; `update`(-1); `store` x
`if` $x = 0$ `then` P_1 `else` P_2	`load` x; `branch`$(= 0?, \hat{P}_1, \hat{P}_2)$
`while` $x \neq 0$ `do` P	`loop`(`load` x; `branch`$(\neq 0?, \hat{P}, \epsilon)$)
$P_1; P_2$	$\hat{P}_1 \circ \hat{P}_2$ (sequential composition)

where $\hat{P}$ denotes the compilation of P. The no-op `skip` compiles to the identity instruction ϵ; the `while` row's trailing `branch` exits to ϵ when the guard fails, which is exactly the unfolding identity below.

A.4 Simulation Lemma

Lemma A.1. For every WHILE configuration $\langle P, \sigma \rangle$ and its encoded AgentISA instruction sequence $\hat{P}$ started in state s_σ:

(a) if $\langle P, \sigma \rangle \to \langle P', \sigma' \rangle$, then the AgentISA machine, started at configuration $(\hat{P}, s_\sigma)$ (remaining instruction sequence, Σ-state), reaches the configuration $(\hat{P}', s_{\sigma'})$—remaining sequence $\hat{P}'$ the encoding of the residual program P', state $s_{\sigma'}$ the encoding of σ'—in finitely many steps; and

(b) if $P =$ `skip` (terminal), then $\hat{P}$ terminates.

Proof. By structural induction on P:

- **Assignments** (base cases). Each compiled sequence performs exactly the corresponding store update; (a) holds by executing the two or three instructions in order, (b) trivially.
- **Conditional.** The `branch` instruction evaluates the zero test on the loaded value and transfers control to one of the compiled branch bodies. By the induction hypothesis applied to P_1 and P_2, each branch simulates its WHILE counterpart.
- **Loop.** The WHILE unfolding identity `while x ≠ 0 do P ≡ if x = 0 then skip else (P ; while x ≠ 0 do P)` is simulated by the `loop` instruction's iteration: each iteration loads x and executes the trailing `branch`, which dispatches to $\hat{P}$ when $x \neq 0$ and to ϵ (loop exit) when $x = 0$. By an outer induction on the number of executed iterations (in addition to the structural induction on P announced above—the induction hypothesis applied to P covers each body execution, and the trailing branch discharges the loop condition)—the k-th WHILE iteration is matched by the k-th `loop` iteration.
- **Sequence.** $\hat{P}_1 \circ \hat{P}_2$ executes $\hat{P}_1$ to completion, then $\hat{P}_2$; both simulations compose by the induction hypothesis.

The case arguments above are the substantive reduction steps; expanding each into a fully formal induction over the small-step rules (matching each WHILE step against the transition functions $\delta \in \Delta$ of the corresponding AgentISA instructions) is mechanical and follows the template of textbook simulation arguments [33, 43]. We mark those expansions as routine and leave them to the reader. ■

A.5 Conclusion

By Lemma A.1, every WHILE-computable function is AgentISA-computable. Since WHILE is Turing equivalent—a classical result of computability theory, with WHILE serving as the standard imperative model in semantics texts such as [43]—AgentISA is Turing complete. ■

A.6 Scope Remark

Turing completeness is a deliberately weak property—any language with unbounded storage, branching, and unbounded iteration attains it. Its role here is negative certification: the instruction set does not artificially constrain what agent programs can express, so no expressiveness objection can be raised against the architecture. The claim says nothing about efficiency, and it is established without the LLM-interaction class $\mathcal{L}$, so it is conservative with respect to the full instruction set of Definition 3.1.

B Proof of Theorem 3.2 (Type Safety)

We prove type safety for the pure core by progress and preservation, in the style of Pierce [36]. The proof runs over a **merged judgment system**: the term typing judgment $\vdash e : T$ for λ-terms, and the instruction judgment $\vdash i\ ok[pre(i), post(i)]$ for instructions and their compositions (both defined in B.1). Progress and preservation are each proved by induction over the derivations of this merged system: the λ-cases are the standard TAPL argument on term derivations; the instruction cases are discharged by the standing totality assumption and the per-operator premises of B.1(ii), with a joint runtime invariant threaded through the composition cases.

B.1 Core Calculus

Restrict attention to the pure core of the instruction language: expressions over base types, dependent Π/Σ pairs, recursion (μ), λ-abstraction and application (whose β-reduction rule appears in Section 3.1.2), and the composition operators of Definition 3.6 applied to instructions carrying typed pre/post-conditions. The small-step relation $\to$ is generated by β-reduction, the context rule $E[e_1] \to E[e_1']$, and, for each instruction i, its **fixed per-instruction** transition function δ_i. Effectful instructions are treated in B.7.

The two judgment forms:

(i) **Term typing** $\vdash e : T$—the usual rules over base types, extended with the dependent Π/Σ introduction forms of Definition 3.2.

(ii) **Instruction typing** $\vdash i\ ok[pre(i), post(i)]$—read "$i$ is a well-contracted state transformer": $pre(i)$ and $post(i)$ are well-typed predicates over the state type, and the Definition 3.5 obligation $\forall s \vDash pre(i) : \delta_i(s) \vDash post(i)$ is discharged per instruction, offline, by the bounded verification of Section 3.3.3. Instructions are **not values and never stuck**: totality on precondition-satisfying states is a standing assumption—for each instruction i with its transition function δ_i, $\delta_i(s)$ is defined whenever $s \vDash pre(i)$.

Composition rules, for instructions i_1, i_2 carrying contracts:

$$\frac{\vdash i_1\ ok \qquad \vdash i_2\ ok \qquad post(i_1) \Rightarrow pre(i_2)}{\vdash i_1 \circ i_2\ ok[pre(i_1),\ post(i_2)]}$$

$$\frac{\vdash i_1\ ok \qquad \vdash i_2\ ok \qquad pre(i_1) \Leftrightarrow pre(i_2) \quad \text{disjoint state footprints}}{\vdash i_1 \parallel i_2\ ok[pre(i_1),\ post(i_1) \wedge post(i_2)]}$$

$$\frac{\vdash i_1\ ok \qquad \vdash i_2\ ok \qquad post(i_1)\big|_{\text{trigger}} \Rightarrow pre(i_2)}{\vdash i_1 \rhd i_2\ ok[pre(i_1),\ post(i_1) \wedge (\text{trigger} \Rightarrow post(i_2))]}$$

$$\frac{\vdash i\ ok \qquad \text{guard closed under } post(i) \qquad \text{guard} \Rightarrow pre(i)}{\vdash i^*\ ok[\text{guard},\ post(i)]}$$

Two premises deserve note. The $\rhd$ premise is evaluated **on the outcome of i_1 that triggers i_2**: i_1's execution can otherwise destroy $pre(i_2)$, so the implication the runtime invariant needs is $post(i_1)$ restricted to the triggering outcome, not $pre(i_1) \Rightarrow pre(i_2)$ on the initial state. The i^* premise adds guard $\Rightarrow pre(i)$ because the loop body re-enters: on every re-iteration the guard holds, so the body's precondition must hold whenever the loop has not terminated. These rules are generated from—do not add to—the semantics of Definition 3.6.

B.2 Canonical Forms

If $\vdash v : T$ and v is a value, then:

- $T \in \{Bool, Int, String, Float\}$ implies v is a literal of that type;
- $T = \Pi(x : T_1).T_2$ implies $v = \lambda x{:}T_1.\, e$ with $\Gamma, x{:}T_1 \vdash e : T_2$;
- $T = \Sigma(x : T_1).T_2$ implies $v = (v_1, v_2)$ with the components typed at the corresponding dependent types;
- $T = \mu X.T$ implies v is a rolled recursive value;
- no instruction or instruction composition is a value (B.1(ii)): instructions are contracted state transformers, not term-level values.

Proof. By inspection of the typing rules: each type has exactly one value form, and the syntax of values is defined per type, so the case analysis is exhaustive. This is the standard canonical-forms lemma [36]. ■

B.3 Lemma (Substitution)

If $\Gamma, x{:}T' \vdash e : T$ and $\vdash v : T'$, then $\Gamma \vdash e[x := v] : T$.

Proof. By induction on the derivation of $\Gamma, x{:}T' \vdash e : T$. The non-dependent cases are the case analysis of Pierce [36], which we do not duplicate. The dependent Π/Σ cases additionally require synchronizing the substitution into the second component's type ($T_2[x := v]$), as in the β-case of B.5—the standard dependent-type substitution bookkeeping, not covered by the non-dependent treatment in [36]. ■

B.4 Lemma (Progress)

If $\vdash e : T$, then e is a value or there exists e' with $e \to e'$.

Proof. By induction on the derivation. The λ-term cases:

- **Literals and** λ are values.

- **Application** $e_1\, e_2$ with $e_1 : \Pi(x : T_1).T_2$ and $e_2 : T_1$: by the induction hypothesis, either e_1 steps (then the context rule applies) or e_1 is a value, in which case B.2 gives $e_1 = \lambda x.\, e$; similarly e_2 either steps or is a value; when both are values, β-reduction applies. There is no stuck case, because the Π-type of e_1 and B.2 jointly guarantee a λ in value position.

The instruction cases admit no stuck state by construction. Instructions step by the standing totality assumption of B.1(ii) whenever their precondition holds. Compositions maintain the runtime invariant **each reached component's precondition holds**, per operator:

- **Sequencing $\circ$:** $pre(i_1)$ holds initially (the composition's contract); after i_1 executes, $post(i_1) \Rightarrow pre(i_2)$ re-establishes the invariant for i_2.
- **Parallel $\|$:** disjoint state footprints keep each component's precondition valid throughout the interleaving—neither component writes what the other's precondition or postcondition reads.
- **Guarded $\triangleright$:** on the triggering outcome of i_1, the strengthened premise $post(i_1)\big|_{\text{trigger}} \Rightarrow pre(i_2)$ re-establishes $pre(i_2)$; on the non-triggering outcome, i_2 does not run.
- **Iteration i^*:** the body re-enters only while the guard holds, and the premise guard $\Rightarrow pre(i)$ makes each re-entry legitimate; when the guard fails, the loop is terminal.

The invariant is threaded jointly with B.5: each individual step preserves the invariant by the per-operator argument above. For iteration, either the guard is false (terminal) or the body's precondition holds by guard $\Rightarrow pre(i)$, and the body steps by the induction hypothesis on the component derivation. ■

B.5 Lemma (Preservation)

If $\vdash e : T$ and $e \to e'$, then $\vdash e' : T$.

Proof. By induction on the derivation, with case analysis on the step:

- **β:** $(\lambda x.\, e)\, v \to e[x := v]$. The application rule types the redex at $T_2[x := v]$; by B.3, the substituted term $e[x := v]$ types at $T_2[x := v]$, which is exactly the type the application rule assigned (the dependent-type bookkeeping; when x does not occur in T_2 this degenerates to the usual $T_2 = T$).
- **Context:** $E[e_1] \to E[e_1']$ with $e_1 \to e_1'$. By the induction hypothesis $e_1' : T_1$; an induction on the structure of E re-types $E[e_1']$ at the original type.
- **Instruction transitions:** the instruction's *ok* judgment (B.1(ii)) certifies that δ_i establishes $post(i)$ from any $pre(i)$-satisfying state (discharged offline by the bounded verification of Section 3.3.3), so the reduct re-types at the postcondition.
- **Composition operators:** each operator's step is typed by its B.1(ii) rule; the reduct's contract follows from the operator's stated conclusion (for $\circ$, $post(i_2)$; for $\|$, the conjunction; for $\triangleright$, the trigger-restricted conjunction; for i^*, $post(i)$), by the induction hypothesis on the component derivations.

The remaining instruction-class cases repeat this pattern with class-specific invariants; they are routine and omitted (the template is the standard preservation argument for the corresponding constructs [36]). ■

B.6 Theorem 3.2

By B.4, a well-typed closed term is a value or takes a step; by B.5, every step preserves typing. Hence no well-typed term reaches a stuck state: well-typed programs in the pure core cannot go wrong. ■

B.7 Scope Remarks

Two caveats state the theorem's boundaries honestly. First, B.4's appeal to totality of transition functions on precondition-satisfying states is an assumption about instruction *implementations*, not something the type system alone discharges. The per-instruction mechanism intended to discharge it is the bounded verification of Section 3.3.3, which is now shipped code: the checker mechanically discharges the obligation for pure instructions on the state spaces it can enumerate—exhaustively for small enumerable domains, by deterministic bounded sampling beyond—while side-effectful instructions and state spaces beyond the enumeration bound remain covered by hand argument and unit tests rather than by the mechanical check. The theorem is conditional on the obligation being (or becoming) true of each instruction, to the extent stated. Second, effectful instructions (LLM calls, resource allocation) can fail at runtime for reasons outside the type system (network errors, exhausted capacity); Theorem 3.2 covers the pure core and effectful instructions with checkable preconditions. Runtime failure handling for the remaining cases is the responsibility of the fallback machinery of Section 5.6, not of types.

C Proofs of Theorems 3.4–3.6

C.1 Denotational Setup

Fix the state space Σ (Definition 3.1's state space—not the dependent-type constructor Σ of Definition 3.2). For a pure instruction i, let $[\![i]\!] : \Sigma \rightharpoonup \Sigma$ denote its denotation, a partial function total on states satisfying $pre(i)$. Program denotation is defined by structural recursion on the composition algebra of Definition 3.6:

- $[\![i]\!]$ for atomic instructions (given);
- $[\![p_1 \circ p_2]\!] = [\![p_2]\!] \circ [\![p_1]\!]$ (note the order: p_1 executes first);
- $[\![p_1 \parallel p_2]\!](\sigma) = [\![p_2]\!]([\![p_1]\!](\sigma))$ for pure p_1, p_2 with disjoint state footprints (as the $\parallel$-typing rule requires)—the order is immaterial because updates to disjoint cells commute on their domains;
- $[\![p_1 \rhd p_2]\!](\sigma) = [\![p_2]\!]([\![p_1]\!](\sigma))$ when p_1's outcome at σ is the trigger, and $[\![p_1]\!](\sigma)$ otherwise (the denotation is defined on $\Sigma +$ outcome, with p_2 applied conditionally on the trigger tag);
- $[\![p^*]\!]$ is the least fixed point of the corresponding functional.

Write $p \equiv q$ iff $[\![p]\!] = [\![q]\!]$ on all states where both are defined.

C.2 Lemma (Atomic Rewrite Soundness)

Each rewrite rule implemented by the optimizer (Section 3.4.4) preserves denotation.

(i) Fusion. $i_1 \circ i_2 \to i_{12}$, with $pre(i_{12}) = pre(i_1)$, $post(i_{12}) = post(i_2)$, and i_{12}'s body stipulated to compute $[\![i_2]\!] \circ [\![i_1]\!]$ (intermediate bindings named, so the composition is explicit). The denotational claim is then an instance of the definition of $\circ$ (C.1): a body computing $[\![i_2]\!] \circ [\![i_1]\!]$ denotes exactly $[\![i_1 \circ i_2]\!]$ (the associativity of function composition is what fuses *three or more* consecutive

instructions). **Two honest boundaries.** First, this is an equality *of the rewrite algebra*: it holds of the fused form whose body computes the composite. The shipped *formal compiler* (the static path of the Section 5.3 benchmarks) produces a formal fused *node* (merged metadata and dependencies) without generating an executable composite body—code generation for fused nodes is not implemented—so what those benchmarks measure is the structural fusion of the formal stream, and the executable-fusion claim would additionally require the codegen step. The *Instruction-level* merge path (the dynamic optimizer of Sections 4.2 and 5.4) does carry an executable body—the `CompositeInstruction` replay construction—and its differential property (composite execution equals constituent-sequence execution; the fusion gate refuses producer–consumer pairs) is pinned by unit tests and exercised by the trace-replay identity of Section 5.4 (5/5 seeds). A fused *single* executable body distinct from replay remains specified. Second, the compiler applies fusion only when output types of i_1 match input types of i_2 and both members are pure, since denotational equality of the fused form presupposes order-insensitivity of intermediate effects, and the shipped gate achieves it via purity (the pure or order-insensitive alternative is specified, not implemented).

(ii) Dead-code elimination. Instruction i is removed from p when $Def(i) \cap Live_{out}(i) = \emptyset$, with liveness the standard fixpoint of $Live_{in}(i) = Use(i) \cup (Live_{out}(i) - Def(i))$ [33]. Claim: if i is dead and side-effect-free, then $p \equiv p \setminus \{i\}$. Proof sketch: by induction on the suffix program following i, its denotation depends only on the live variables at i's position; i writes none of them, so removing i cannot change any value subsequently read. The side-effect-freeness gate matches the purity restriction of Definition 3.6 (Property 2 and the Remark following it)—instructions declared `side_effects=True` are never eliminated.

(iii) Peephole combining. Each rule $(s_1, \ldots, s_k) \to s'$ carries the verification obligation $\forall \sigma : [\![s_1 \circ \ldots \circ s_k]\!](\sigma) = [\![s']\!](\sigma)$, to be discharged at rule-definition time (Section 4.1.3). Candidate instances, *illustrative only—the shipped optimizer registers none of them*: `LOAD A; STORE B` composes to `MOVE A→B` (degenerate to the identity when A = B, since the value read is the value written; the general rule of Section 3.4.4); consecutive `QUERY` instructions with identical (instruction, params) pairs collapse to one query whose result is reused (the batch-QUERY rule of Section 3.4.4, specialized to identical pairs) (an obligation about a *compile-time* rewrite; the caching measured in Section 5.2 is a different, *runtime* mechanism—a result cache consulted at execution time—and its numbers do not evidence this rule); consecutive `CONTEXT_UPDATE`s compose to a single update by associativity of the update function.

Proof. Each case reduces to the construction of the rewritten instruction and the definition of liveness. The inner inductions—on the pair of bodies for (i), on suffix programs for (ii)—are routine and left to the reader. ■

C.3 Lemma (Compositionality)

Let T be a program transform with $[\![T(p)]\!] = [\![p]\!]$ for all p. Then for any context $C[\cdot]$ built from the composition operators, $[\![C[T(p)]]\!] = [\![C[p]]\!]$.

Proof. By structural induction on C. Each operator's denotation (C.1) is defined compositionally in its constituents' denotations, so substituting a denotationally equal constituent leaves the whole unchanged. The parallel case uses the disjoint-footprint hypothesis of the $\|$-typing rule (updates to disjoint cells commute). The iteration case uses the defining property of least fixed points: if the functionals induced by p and $T(p)$ are equal, then all finite approximants are equal (by the induction hypothesis), so their suprema—the least fixed points—are equal. ■

C.4 Theorem 3.4 (Optimization Soundness)

Proof. Read all equalities below as the partial-equivalence $\equiv$ of C.1 (equality on all states where both sides are defined), since the DCE rule preserves denotation only on the defined domain and can strictly enlarge it. The compiler applies a finite sequence of rewrites $O = R_k \circ ... \circ R_1$ (finiteness follows from compilation terminating; each pass is a bounded traversal). By induction on k: $k = 0$ is trivial. For the step, write $O = R \circ O'$. By the induction hypothesis $[\![O'(p)]\!] \equiv [\![p]\!]$. Two routes, by rule kind. **Fusion and peephole** are local subprogram rewrites: the atomic-rewrite lemma (C.2) preserves the denotation of the subprogram it rewrites, and the compositionality lemma (C.3) lifts the preservation through any surrounding composition context. **Dead-code elimination** is not local—liveness is computed over the whole surrounding program—so it is handled at pass level directly by C.2(ii)'s whole-program argument ($p \equiv p \setminus \{i\}$), not through the lifting template. Hence $[\![R(O'(p))]\!] \equiv [\![O'(p)]\!] \equiv [\![p]\!]$. ■

C.5 Theorem 3.5 (Pipeline Correctness)

Let s be a valid instruction sequence and S a schedule satisfying:

- **(S1)** every instruction is dispatched exactly once;
- **(S2)** for every dependency $(i_1, i_2) \in \mathit{Deps}$: $\mathit{start}(i_2) \geq \mathit{end}(i_1)$ (the constraint of Section 3.4.3);
- **(S3)** instructions sharing a parallel dispatch group are pairwise non-interfering (no data hazard, no resource conflict);
- **(S4)** commit is in program order.

Then executing s under S yields the same final state as sequential execution: $[\![s]\!]_S = [\![s]\!]_{seq}$.

Proof. Let $g_1, ..., g_m$ be the dispatch groups of S in commit order. By S4, groups commit in program order and group commits do not interleave, so the union $\bigcup_{j \leq t} g_j$ of committed groups is always a program-order prefix—"executing all instructions in those groups" below means executing that prefix sequentially. We show the invariant

$$I(t) : \text{after committing } g_1, ..., g_t,$$
$$\text{the architectural state equals the sequential state after}$$
$$\text{executing all instructions in those groups.}$$

- **Base** ($t = 0$): both sides are the initial state.
- **Step**: assume $I(t)$. Every instruction in g_{t+1} has its dependencies in earlier groups (by S2, since it is scheduled after them), which by $I(t)$ have completed and committed. Within the group, S3 gives pairwise non-interference: no instruction reads another group-member's output, and no two write the same location or resource. Hence the group's updates commute—for pure updates this is the diamond property, and for effectful instructions, S3's exclusion of shared resources makes their effects independent, so the combined effect is unique regardless of interleaving. The state after g_{t+1} is therefore independent of the group's internal interleaving and equals the sequential execution of the same instructions from the state at $I(t)$.
- By induction, $I(m)$ holds; by S1 no instruction was skipped or duplicated.

This completes the induction over all dispatch groups, hence the proof. ■

Corollary (dispatch folding). The ADVANCED and AGGRESSIVE levels fold instructions into shared dispatches (Section 5.3). A folded group satisfies S3 by construction: the grouping check

admits only side-effect-free instructions whose dependencies and declared outputs are pairwise disjoint, so the group's members act on disjoint state components and their updates commute (Definition 3.6, Property 2). By Theorem 3.5 the fold changes the *dispatch count* but neither the semantics nor—consistent with the measurement of Section 5.3—the effective instruction count.

C.6 Theorem 3.6 (Pass-Pipeline Soundness)

Every optimization in the pass pipeline is a finite composition of rewrites covered by the atomic-rewrite lemma (C.2) or by the dispatch-folding corollary of Theorem 3.5 (C.5)—the grouping pass is a fold, not a local rewrite—and the argument of C.4 applies verbatim to each rewrite layer. ■

C.7 Scope Remark

Theorems 3.4–3.6 are *conditional*: they hold of transformations whose verification obligations (Sections 4.1.3 and 4.3.1) are discharged. As stated there, those obligations are **specified but not yet mechanically discharged**—the shipped optimizer registers no peephole rules (the peephole obligation is dormant, and the fusion/DCE/grouping passes are argued directly in C.2 and C.5 rather than through the symbolic-execution checker), and the symbolic-execution equivalence checker itself is a specification (Section 4.3.1). Where an obligation is discharged only on bounded state spaces (Section 3.3.3), the guarantee is correspondingly bounded: rewrites verified on the explored subset are sound there. No such rewrite is registered today; when one is, its application must be restricted to the explored subset (that restriction is a dormant obligation of the future rule set, not shipped behavior). What the shipped implementation does provide is empirical semantics-preservation at task level: the system-level demonstration of Section 5.8 re-executes an optimized stream and checks byte-identical output.

Observation model. The theorems' equality is *partial equivalence* (C.1: $[\![p]\!] = [\![q]\!]$ on states where both are defined), and the observation model should be read accordingly. **Observed:** the final state of declared state cells, the declared outputs (the instruction's product keys), and—for theorems covering effectful instructions (C.5)—the externally visible effects, in program order. **Deliberately not observed:** the *domain of definition* itself. Because equivalence is only partial, a rewrite may extend definedness—DCE is the canonical example (C.4): deleting an instruction that would have failed can turn a previously undefined run into a defined one. If program failure or exception behavior were part of the observation model, this would violate preservation; it is therefore stated explicitly that failure behavior is outside what Theorems 3.4–3.6 preserve, and any deployment for which failures are observable (e.g., a pipeline contract that treats exceptions as data) needs a strengthened equivalence and must re-derive the theorems under it. Internal traces—the sequence of instructions executed—are likewise not observed; only the net transformation is.

Three status levels. Each mechanism in this paper falls into exactly one of three levels, which we keep separate throughout: *(i) specified*—the symbolic-execution equivalence checker, the pipeline scheduler's cycle-accurate extension, tree-pattern instruction selection (Section 4.1.2); *(ii) implemented with checks*—the optimizer passes (fusion, DCE, grouping) with their pass-level arguments in C.2/C.5, the runtime monitors, and differential unit tests pinning the fusion property this appendix argues (fused-composite execution equals constituent-sequence execution; the fusion gate refuses producer–consumer pairs; `tests/core/isa/test_fusion_differential.py`); *(iii) empirically validated*—trace-level replay identity on the mock (Section 5.4: optimized traces re-executed, final states identical in 5/5 seeds), task-level re-execution with byte-identical outputs (Section 5.8), and the caching/compilation measurements of Section 5 (including the exact-response-cache parity

baseline of Section 5.2). A statement at one level is never used to imply a guarantee at a stronger level.